DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 – DATA WAREHOUSING AND DATA MINING

ASSIGNMENT 5 – CO5

Design and Evaluate an Integrated Data Warehousing and Mining System for a Smart-City Citizen Services Platform

CO5 Coverage: Text mining; information retrieval; web mining; automatic classification of web documents; spatial data mining; spatial association and correlation patterns; spatial clustering methods; social impacts of data mining; and design of data warehousing and data mining systems for real-world applications.

Field	Details	Field	Details
Student Name	A. Mohith Krishna	Register Number	192425199
Department / Section	CSE	Academic Year / Semester	Oct 2026
Faculty Name	Dr. KV. Kanimozhi	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	05-11-2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Task Title	Max Marks	Awarded Evidence / evaluation focus
1	Text Data Analysis and Information Retrieval	5	Preprocessing; term-frequency matrix; IDF and TF–IDF; cosine similarity; precision and recall; limitations
2	Text Mining Implementation	6	Corpus; feature settings; classifiers; accuracy; confusion matrix; clustering; misclassification analysis
3	Web Mining: Structure, Usage and Page Importance	5	Mining categories; PageRank iterations; hubs and authorities; DOM and page layout
4	Automatic Classification of Web Documents	5	Pipeline; web-specific features; challenges and remedies; evaluation and retraining
5	Spatial Data and Spatial Association Patterns	6	Spatial types and relationships; autocorrelation; participation ratios and indices; prevalence
6	Spatial Clustering of Incident Locations	5	Dataset; k-means; DBSCAN; k-distance graph; plots; noise and method comparison
7	Social Impacts and Privacy	4	k-anonymity; generalisation; information loss; social risks and safeguards
8	Design the Integrated System	6	Architecture; dimensional model; technique mapping; privacy controls; evaluation plan
9	Day-4 Design Challenge	8	Impact analysis; evaluation; revised design; evidence-based defence
	TOTAL	50	Percentage:

EVIDENCE VERIFICATION CHECKLIST

Task	Evidence expected	Status (✓ / P / ✗)	Evaluator note
1	Preprocessing terms, term matrix, IDF/TF–IDF, cosine ranking and precision/recall		
2	Corpus description, feature-setting comparison, classifier metrics, confusion matrix and clustering		
3	Web mining categories, two PageRank iterations, hub/authority explanation and layout discussion		
4	Classification pipeline, web-specific feature table, challenge-remedy mapping and evaluation plan		
5	Spatial data examples, relationship types, participation ratio/index calculations and co-location decision		
6	At least 1,000 GPS points, k-distance graph, cluster plots, noise identification and comparison		
7	k-anonymity calculation, 2-anonymous generalised table and risk–safeguard table		
8	Integrated architecture, fact/dimension design, technique mapping, privacy controls and evaluation		
9	Impact analysis, revised design and evidence supporting privacy/accuracy trade-offs		

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Observation / evidence to be attached	Status
Source code / workflow	Text-mining and spatial-clustering code or workflows, with comments and settings	
Dataset	Corpus and GPS incident dataset (real or synthetic), with source and description	
Feature extraction	Vectorisation settings, vocabulary configuration and preprocessing details	
Model execution	Classifier training, k-means and DBSCAN execution evidence	
Functional evaluation	Accuracy, confusion matrix, precision/recall and clustering interpretation	
Visual evidence	PageRank calculations, k-distance graph, cluster plots, tables and diagrams	
Reproducibility	Software versions, parameter values, random seeds and execution steps	

OVERALL REMARKS

_____

_____

_____

Student Signature: _____ Faculty Signature: _____ Program Director: _____

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = *Present in documented answer; P = Partly present;* ✗ = *Missing / not satisfied*

Task	Evidence expected	Status	Evaluator note
1	Text preprocessing, TF–IDF calculations, cosine similarity and retrieval metrics		
2	Text corpus, feature experiments, classification and clustering results		
3	PageRank working, web mining concepts and page-layout explanation		
4	Web-document classification pipeline and evaluation strategy		
5	Spatial data concepts and complete co-location calculations		
6	GPS dataset, parameter selection, plots and cluster interpretation		
7	Privacy analysis, k-anonymity and safeguards		
8	Integrated data warehouse/mining architecture and dimensional design		
9	Impact analysis, redesign/adaptation and evidence-based defence		

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All Tasks 1–9 answered	☐ Yes ☐ Partly ☐ No
Hand calculations shown for TF–IDF, PageRank, participation index and k-anonymity	☐ Yes ☐ Partly ☐ No
Text-mining and spatial-clustering source code/workflows attached	☐ Yes ☐ Partly ☐ No
Datasets, parameter settings and random seeds documented	☐ Yes ☐ Partly ☐ No
Tables, plots and labelled architecture diagrams included	☐ Yes ☐ Partly ☐ No
Privacy controls and design challenge discussed	☐ Yes ☐ Partly ☐ No
Figures and tables numbered correctly	☐ Yes ☐ Partly ☐ No
Viva / verification discussion recommended	☐ Yes ☐ No

CSA16 – DATA WAREHOUSING AND DATA MINING

ASSIGNMENT 5 – CO5

Course Outcome: CO5 – Describe and utilise a range of techniques for designing data warehousing and data mining systems for real-world applications.

Assignment Title: Design and Evaluate an Integrated Data Warehousing and Mining System for a Smart-City Citizen Services Platform (Total Marks – 50)

Nature: Individual | Duration: 1 Week

Scenario – Metro City Citizen Services Analytics

Metro City Corporation serves 4 million residents. Every day it receives about 5,000 citizen complaints (free text from a mobile app, portal and social media), publishes service information on its web portal, records road-accident and incident locations with GPS coordinates, and stores completed service requests in an operational database.

The Commissioner wants a single analytics platform that can:

- Retrieve and group complaints by topic and urgency.
- Understand how citizens use the web portal and classify its pages and external news pages.
- Find geographic hot-spots of incidents, and relationships between spatial features.
- Support summary reports through the data warehouse.
- Do all this without violating the privacy of citizens.

You are required to investigate suitable techniques for each need and to design an integrated system.

Design Challenge

A new data-protection rule prohibits storing exact residential locations and personal identifiers in the analytical platform, and complaints now arrive in three languages. Re-evaluate your design and determine what must be retained, modified or redesigned.

TASK 1 – Text Data Analysis and Information Retrieval (5 Marks)

- The following four complaints form a mini document collection:
- D1: pothole on main road near school
- D2: garbage not collected near school road
- D3: street light not working on main road
- D4: water leakage near school gate
- A user enters the query “school road garbage”. Users judge D1 and D2 as relevant.
- Describe and apply preprocessing steps (tokenisation, stop-word removal, stemming) and list the resulting terms.
- Build the term-frequency matrix and compute IDF = log10(N/df) and TF–IDF weights.
- Rank the documents for the query using cosine similarity in the vector-space model, showing calculations.
- Compute precision and recall at rank 2 and explain what they reveal about the ranking.
- Explain one weakness of keyword matching for complaint text (such as synonyms and polysemy) and one remedy.

Evidence Expected: Term matrix, IDF/TF–IDF table, cosine-similarity ranking, precision/recall and a short discussion of limitations. Show every intermediate step.

TASK 2 – Text Mining Implementation (6 Marks)

- Use a text corpus of at least four categories (for example a subset of 20 Newsgroups, or documented real/synthetic complaints).

- Convert the text to a vector representation (for example Weka StringToWordVector or Python TF-IDF).
- Compare at least three feature settings (for example binary vs TF–IDF weights, with/without stop-word removal and stemming, 500 vs 5,000 vocabulary size).
- Train at least two classifiers (for example Naïve Bayes and SVM) and report accuracy, a confusion matrix and per-class precision/recall.
- Cluster unlabelled documents with k-means and examine whether clusters agree with the categories.
- Examine misclassified documents and explain why they were misclassified.

Evidence Expected: Experimental tables, confusion matrix, clustering comparison and analysis of misclassified documents.

TASK 3 – Web Mining: Structure, Usage and Page Importance (5 Marks)

- The Corporation's portal has five pages with links: H → S, C; S → C; C → H; N → H, C, A; A → N (H = Home, S = Services, C = Complaints, N = News, A = Archive).
- Distinguish web content mining, web structure mining and web usage mining, and give one use of each in this scenario.
- Compute PageRank for two iterations with damping factor d = 0.85, starting with equal ranks of 0.2, showing every step. Identify the most important page and explain why.
- Explain hubs and authorities and identify which pages might be hubs.
- Explain web page layout structure (such as the DOM tree and block segmentation) and why navigation menus and advertisements should be separated from main content before mining.

Evidence Expected: Web mining categories with examples, PageRank iterations, hub/authority explanation and layout-structure discussion.

TASK 4 – Automatic Classification of Web Documents (5 Marks)

- The Corporation wants to classify web pages (portal pages and linked news pages) into Roads, Water, Sanitation, Electricity, Health and Other.
- Design the complete pipeline: collection → cleaning (removal of tags, scripts and boilerplate) → feature extraction → training → evaluation → deployment.
- Identify web-specific features (for example title, headings, URL tokens, anchor text and hyperlinks from neighbouring pages) and explain how each helps.
- Explain challenges of web data (noise, multi-topic pages, dynamic content and class imbalance) and one remedy for each.
- State how you would evaluate the classifier and detect when it needs retraining.

Evidence Expected: Labelled pipeline diagram, feature table with justification and evaluation plan.

TASK 5 – Spatial Data and Spatial Association Patterns (6 Marks)

- Describe the main spatial data types (point, line, polygon, raster) and spatial relationships (topological, distance, direction), with examples from city data.
- Explain spatial autocorrelation and why standard statistical methods that assume independent observations can mislead on spatial data.
- Neighbouring feature instances are: A (accident spots) = 4 instances; B (bars) = 5 instances; C (cross-road junctions without lights) = 3 instances.
- A–B neighbours: (A1,B1), (A1,B2), (A2,B2), (A3,B4).
- A–C neighbours: (A1,C1), (A2,C1), (A3,C2).
- B–C neighbours: (B1,C1), (B2,C1).

- For each pair, compute the participation ratio of each feature and the participation index.
- With a minimum prevalence threshold of 0.5, identify prevalent size-2 co-locations.
- The only instances forming the triple are (A1,B1,C1), (A1,B2,C1) and (A2,B2,C1). Compute the participation index of {A, B, C} and decide whether it is prevalent.
- Explain why a co-location is not the same as a market-basket association rule.

Evidence Expected: Spatial data and relationship examples, participation ratio/index calculations and correct decision on prevalent co-locations.

TASK 6 – Spatial Clustering of Incident Locations (5 Marks)

- Use real or synthetic GPS incident data (at least 1,000 points) containing dense hot-spots of different shapes and scattered noise points. Document the source.
- Run k-means and DBSCAN (and discuss, without implementing, how a medoid-based method such as CLARANS would differ).
- Select DBSCAN parameters (ε and MinPts) using a k-distance graph and show the effect of changing ε.
- Plot the clusters and identify points treated as noise.
- Explain which method better represents the hot-spots and why, based on how each method defines a cluster.

Evidence Expected: Plots, parameter-selection evidence and an interpreted comparison of methods.

TASK 7 – Social Impacts and Privacy (4 Marks)

- The following extract is to be released to researchers:
- Age / PIN code / Complaint type: 23 / 600041 / Garbage; 27 / 600042 / Garbage; 24 / 600041 / Water; 52 / 600087 / Roads; 58 / 600088 / Roads; 55 / 600087 / Water.
- Treating Age and PIN code as quasi-identifiers, determine the k-anonymity of the table as given.
- Generalise the quasi-identifiers (for example age ranges and PIN-code prefixes) to achieve 2-anonymity, and state what information is lost.
- Identify three social risks of mining the platform's data (for example profiling, bias against neighbourhoods, misuse) and a safeguard for each.

Evidence Expected: k-anonymity analysis, anonymised table and risk–safeguard table.

TASK 8 – Design the Integrated System (6 Marks)

- Design the integrated warehousing and mining platform for the Commissioner.
- Include a labelled architecture showing data sources, ETL (including how unstructured text and spatial data are loaded), warehouse, mining modules and reporting layer.
- Provide a dimensional design (fact and dimensions) for service requests, including a location dimension and a time dimension.
- Map each requirement of the Commissioner to the technique you would use and give the reason.
- Include privacy controls built into the design and explain how you will evaluate whether each module is working.

Evidence Expected: Architecture diagram, dimensional design, requirement-to-technique mapping, privacy controls and evaluation plan. A diagram without reasoning will not receive full credit.

DAY-4 DESIGN CHALLENGE – TASK 9: Re-evaluate and Adapt the Design (8 Marks)

- Changed Requirement: A new rule prohibits storing exact residential locations and personal identifiers in the analytical platform. Complaints now arrive in English, Tamil and Hindi, and the daily volume rises to 20,000 complaints.
- Re-evaluate your design. You may retain, modify or redesign parts of the system, and you must justify the decision.

A. Impact Analysis (2 Marks): Identify which modules (ETL, warehouse dimensions, text mining, spatial mining, reports) are affected and which are not.

B. Evaluation of the Original Design (2 Marks): Test the original design against the new conditions, for example the effect of multilingual text on vocabulary/classifier or location generalisation on hot-spot results.

C. Adaptation or Redesign (2 Marks): Present the revised design (for example generalised location hierarchy, anonymisation at ETL, language detection and per-language or multilingual features). Show the changed parts.

D. Evidence-Based Defence (2 Marks): Defend your decision using evidence from your own tasks (for example grid-level location effects on DBSCAN clusters or vocabulary-size effects on accuracy) and state the accuracy or privacy trade-off accepted.

Evidence Expected: Reasoned impact analysis, evaluation, revised design and evidence-based defence.

MARK DISTRIBUTION

Task	Title	Marks
1	Text Data Analysis and Information Retrieval	5
2	Text Mining Implementation	6
3	Web Mining: Structure, Usage and Page Importance	5
4	Automatic Classification of Web Documents	5
5	Spatial Data and Spatial Association Patterns	6
6	Spatial Clustering of Incident Locations	5
7	Social Impacts and Privacy	4
8	Design the Integrated System	6
	Subtotal	42
9	Day-4 Design Challenge	8
	TOTAL	50

SUBMISSION REQUIREMENTS

Submit:

- Your report containing the required evidence for Tasks 1–9.
- Source code/workflows for Tasks 2 and 6.
- Datasets (or links), settings and random seeds.
- Tables, plots and diagrams used in your analysis.

Hand calculations are primary evidence. Tool outputs may be used to verify them but must not replace them.

ASSESSMENT VERIFICATION

Students may be asked to explain any part of their hand calculations (TF–IDF, PageRank, participation index and k-anonymity), system design decisions, source code and tool settings,

experimental results, and the Design Challenge during a brief verification discussion/viva. The student should be able to explain and defend the evidence submitted as their own work.

OVERALL ASSESSMENT EXPECTATION

This assignment does not primarily assess the ability to reproduce textbook definitions or tool outputs. It assesses the ability to:

Model → Select Technique → Compute → Implement → Evaluate → Protect Privacy → Integrate → Adapt

Conclusions and design decisions must be supported by the student's own reasoning and evidence.

DATA WAREHOUSING AND DATA MINING
ASSIGNMENT 5 – CO5

CO5 Coverage: Text mining: text data analysis and information retrieval; web mining: web page layout structure and automatic classification of web documents; spatial data mining: spatial data, spatial association and correlation patterns, spatial clustering methods; social impacts of data mining. Designing data warehousing and data mining systems for real-world applications.

Course Outcome: CO5 – Describe and utilise a range of techniques for designing data warehousing and data mining systems for real-world applications.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Assignment Title: Design and Evaluate an Integrated Data Warehousing and Mining System for a Smart-City Citizen Services Platform (Total Marks – 50)

Scenario — Metro City Citizen Services Analytics. Metro City Corporation serves 4 million residents. Every day it receives about 5,000 citizen complaints (free text from a mobile app, portal and social media), publishes service information on its web portal, records road-accident and incident locations with GPS coordinates, and stores completed service requests in an operational database.

The Commissioner wants a single analytics platform that can:

- Retrieve and group complaints by topic and urgency
- Understand how citizens use the web portal and classify its pages and external news pages
- Find geographic hot-spots of incidents, and relationships between spatial features
- Support summary reports through the data warehouse
- Do all this without violating the privacy of citizens

You are required to investigate suitable techniques for each need and to design an integrated system.

Design Challenge (used in Task 9): A new data-protection rule prohibits storing exact residential locations and personal identifiers in the analytical platform, and complaints now arrive in three languages. Re-evaluate your design and determine what must be retained, modified or redesigned.

How this report is organised

Each task gives the question as set, followed by my answer under **Ans:**. Hand calculations (TF–IDF, cosine similarity, PageRank, participation index, k-anonymity) are shown step by step and are the primary evidence; the programs only verify them or, for Tasks 2, 6 and 9, produce the experiments. All experimental numbers come from three programs, text_mining.py, spatial.py and multilingual.py (Appendices A–C), run with random seed 42; the data sets are **synthetic** and are described in the task where they are used. Figures and tables are numbered in order.

Relation to CO5. The tasks follow the chain Model → Select Technique → Compute → Implement → Evaluate → Protect Privacy → Integrate → Adapt. Tasks 1–2 (text), 3–4 (web), 5–6 (spatial) and 7 (privacy) investigate the individual techniques; Task 8 integrates them into one platform; Task 9 tests that platform against changed requirements using the evidence of Tasks 2, 6 and 7.

TASK 1 – Text Data Analysis and Information Retrieval5 Marks

The following four complaints form a mini document collection (D1: pothole on main road near school; D2: garbage not collected near school road; D3: street light not working on main road; D4: water leakage near school gate). A user enters the query **"school road garbage"**. Users judge D1 and D2 as relevant.

- Describe and apply the preprocessing steps (tokenisation, stop-word removal, stemming) and list the resulting terms
- Build the term-frequency matrix and compute IDF = log10(N/df) and TF–IDF weights
- Rank the documents for the query using cosine similarity in the vector-space model, showing your calculations
- Compute precision and recall at rank 2 and explain what they reveal about the ranking
- Explain one weakness of keyword matching for complaint text (for example synonyms and polysemy) and one remedy

Evidence Expected

Term matrix, IDF/TF–IDF table, cosine-similarity ranking, precision/recall and a short discussion of the limitations of keyword retrieval.

Ans:

1. Preprocessing

Tokenisation splits each complaint into lower-case word tokens at spaces and punctuation. **Stop-word removal** deletes very frequent function words that do not help to tell complaints apart; for this tiny collection I use the stop-list {on, not, near}. **Stemming** reduces each word to a root by stripping suffixes (collected → collect, working → work), so that different forms of a word are counted as one term.

Table 1: Preprocessing of the collection and the query

Doc	After tokenisation	After stop-word removal	After stemming (final terms)
D1	pothole, on, main, road, near, school	pothole, main, road, school	pothole, main, road, school
D2	garbage, not, collected, near, school, road	garbage, collected, school, road	garbage, **collect**, school, road
D3	street, light, not, working, on, main, road	street, light, working, main, road	street, light, **work**, main, road
D4	water, leakage, near, school, gate	water, leakage, school, gate	water, leakage, school, gate
Query	school, road, garbage	school, road, garbage	school, road, garbage

The collection has **12 distinct terms** and N = 4 documents. Removing "not" has a cost: "not collected" and "collected" now look the same, which is discussed in part 5.

2. Term-frequency matrix, IDF and TF–IDF

IDF = log10(N/df) with N = 4: a term in one document has IDF = log10(4) = **0.602**; in two documents log10(2) = **0.301**; in three documents log10(4/3) = **0.125**. TF–IDF = tf × IDF (all raw tf values are 0 or 1 here).

Table 2: Term-frequency matrix, document frequency and IDF

Term	D1	D2	D3	D4	df	IDF = log10(4/df)
collect	0	1	0	0	1	0.602
garbage	0	1	0	0	1	0.602
gate	0	0	0	1	1	0.602
leakage	0	0	0	1	1	0.602
light	0	0	1	0	1	0.602
main	1	0	1	0	2	0.301
pothole	1	0	0	0	1	0.602
road	1	1	1	0	3	0.125
school	1	1	0	1	3	0.125
street	0	0	1	0	1	0.602
water	0	0	0	1	1	0.602
work	0	0	1	0	1	0.602

Table 3: TF–IDF weights (the query vector uses tf = 1 for each query term)

Term	D1	D2	D3	D4	Query
collect	0	0.602	0	0	0
garbage	0	0.602	0	0	0.602
gate	0	0	0	0.602	0
leakage	0	0	0	0.602	0
light	0	0	0.602	0	0
main	0.301	0	0.301	0	0
pothole	0.602	0	0	0	0
road	0.125	0.125	0.125	0	0.125
school	0.125	0.125	0	0.125	0.125
street	0	0	0.602	0	0
water	0	0	0	0.602	0
work	0	0	0.602	0	0

3. Cosine-similarity ranking

cos(d, q) = (d · q) / (|d| |q|). Squares of the weights: $0.602^2 = 0.3625$, $0.301^2 = 0.0906$, $0.125^2 = 0.0156$.

$|q| = \sqrt{(0.602^2 + 0.125^2 + 0.125^2)} = \sqrt{(0.3625 + 0.0156 + 0.0156)} = \sqrt{0.3937} = 0.6275$

D1: d·q = (road) 0.125×0.125 + (school) 0.125×0.125 = 0.0312
|D1| = √(0.602²+0.301²+0.125²+0.125²) = √0.4843 = 0.6959
cos = 0.0312 / (0.6959 × 0.6275) = 0.0312 / 0.4367 = 0.0715
D2: d·q = (garbage) 0.602×0.602 + 0.0156 + 0.0156 = 0.3937
|D2| = √(0.602²+0.602²+0.125²+0.125²) = √0.7563 = 0.8696
cos = 0.3937 / (0.8696 × 0.6275) = 0.3937 / 0.5457 = 0.7215
D3: d·q = (road) 0.0156; |D3| = √1.1937 = 1.0925
cos = 0.0156 / (1.0925 × 0.6275) = 0.0156 / 0.6855 = 0.0228
D4: d·q = (school) 0.0156; |D4| = √1.1031 = 1.0503
cos = 0.0156 / (1.0503 × 0.6275) = 0.0156 / 0.6591 = 0.0237

Table 4: Ranking for the query "school road garbage"

Rank	Doc	d · q	|d|	cos(d, q)	Judged relevant?
1	D2	0.3937	0.8696	**0.7215**	Yes
2	D1	0.0312	0.6959	**0.0715**	Yes
3	D4	0.0156	1.0503	0.0237	No
4	D3	0.0156	1.0925	0.0228	No

4. Precision and recall at rank 2

Retrieved at rank 2 = {D2, D1}; relevant set = {D1, D2}; both retrieved documents are relevant.

Precision@2 = relevant retrieved / retrieved = 2 / 2 = 1.00
Recall@2 = relevant retrieved / all relevant = 2 / 2 = 1.00

What this reveals. The top two places contain exactly the two documents the users wanted, so the **order of the set is correct**. But the scores tell a more cautious story: D2 scores 0.72, while D1 scores only 0.07, barely above the irrelevant D4 (0.024) and D3 (0.023). D1 reaches rank 2 only through the two weak, common words "school" and "road"; it does not contain "garbage". Precision and recall at a single cutoff do not show this thin margin, so a small change of weights (or one more complaint containing "school" and "road") could push an irrelevant document above D1. At rank 1 the values would be P = 1.0 but R = 0.5, which shows how the cutoff decides the picture.

5. Weakness of keyword matching and a remedy

Weakness – vocabulary mismatch (synonyms) and polysemy. Keyword matching only sees identical strings. A citizen who writes "rubbish", "trash" or "waste" scores 0 against the query "garbage", although the complaint is about the same problem; a Tamil or Hindi word for the same thing is invisible as well. Polysemy works the other way: "light" in "street light" (electricity) and "light traffic" (roads) is one term, and my stop-word step turned "not collected" into "collect", losing the negation completely.

Remedy – query expansion with a domain thesaurus (or latent semantic indexing). Before searching, the query is expanded with synonyms from a hand-built complaint thesaurus (garbage ⇄ rubbish, trash, waste, dustbin, kachra) so "garbage" also matches "rubbish". A more automatic remedy is Latent Semantic Indexing: an SVD of the term–document matrix maps words that occur in similar contexts to nearby directions, so synonyms match without sharing a string. Keeping short phrases ("not working", "not collected") as bigram terms reduces the negation and polysemy problems.

TASK 2 – Text Mining Implementation6 Marks

Use a text corpus of at least four categories (for example a subset of 20 Newsgroups, or a set of real or synthetic complaints that you document).

- Convert the text to a vector representation (for example Weka StringToWordVector or Python TF-IDF)
- Compare at least three feature settings (for example binary vs TF–IDF weights, with/without stop-word removal and stemming, 500 vs 5,000 vocabulary size)
- Train at least two classifiers (for example Naïve Bayes and SVM) and report accuracy, a confusion matrix and per-class precision/recall
- Cluster the unlabelled documents with k-means and examine whether the clusters agree with the categories
- Examine the misclassified documents and explain why they were misclassified

Evidence Expected

Experimental tables, confusion matrix, clustering comparison and an analysis of misclassified documents.

Ans:

1. Corpus and set-up

Corpus (synthetic, documented). The 20 Newsgroups data cannot be downloaded in my environment, so I generated **600 synthetic Metro City complaints in 5 categories (Roads, Water, Sanitation, Electricity, Health), 120 each**, with text_mining.py (Appendix A, random seed 42). Each complaint combines one category phrase (for example “pothole”, “water leakage”, “garbage not collected”, “street light not working”, “mosquito breeding”), a location phrase shared by all categories (“near school”, “on main road”) and optional filler (“please fix”, “urgent”). To make the task realistic, **25% of complaints also contain a phrase from a different category**, so some are truly ambiguous. Limitation: because the text comes from templates, absolute accuracies are higher than they would be for real citizen text; I therefore compare settings against each other and do not quote them as real-world accuracy.

Vector representation and split. Python scikit-learn: CountVectorizer(binary=True) and TfidfVectorizer. Stratified split 70% / 30% (420 training, 180 test complaints, 36 per class), random_state = 42. Stop-words removed from a list of 18 function words; stemming by a simple suffix stripper (-ing, -ed, -es, -s). The full corpus contains only 119 distinct raw terms, so a cap of 500 or 5,000 has no effect; I therefore compared the full vocabulary with a vocabulary **capped at 30 terms**.

2. Three feature settings and two classifiers

Table 5: Test accuracy (180 test complaints)

Setting	Features	Vocabulary	Naïve Bayes	Linear SVM
S1	Binary weights, no stop-word removal, no stemming	119	0.8889	0.9333
S2	TF–IDF weights, stop-words removed, stemming	102	0.9111	0.9222

Setting	Features	Vocabulary	Naïve Bayes	Linear SVM
S3	As S2 but vocabulary capped at 30 terms	30	0.4222	0.5278

Reading the table. S1 and S2 differ by only 2–4 test complaints out of 180 (for the SVM 0.933 against 0.922; for Naïve Bayes 0.889 against 0.911), which is within sampling noise, so on this small, template-based corpus neither binary nor TF–IDF weighting, nor stop-word removal and stemming, makes a reliable difference. The **vocabulary size is the strong effect**: cutting the vocabulary from 102 to 30 terms drops the SVM to 0.528 and Naïve Bayes to 0.422, because most category words ("pothole", "mosquito", "transformer") are no longer features at all. I keep **S2** (TF–IDF, stop-words, stemming) as the working setting because it gives a smaller vocabulary and does not depend on the filler words.

3. Confusion matrices and per-class precision and recall (S2, test set)

Table 6: Linear SVM – confusion matrix (accuracy 0.9222, 14 errors)

Actual \ Predicted	Roads	Water	Sanitation	Electricity	Health
Roads	34	1	0	1	0
Water	0	32	0	3	1
Sanitation	0	1	32	2	1
Electricity	0	0	0	35	1
Health	0	1	1	1	33

Table 7: Naïve Bayes – confusion matrix (accuracy 0.9111, 16 errors)

Actual \ Predicted	Roads	Water	Sanitation	Electricity	Health
Roads	33	2	0	1	0
Water	0	33	0	2	1
Sanitation	0	2	32	2	0
Electricity	0	1	2	33	0
Health	1	0	1	1	33

Table 8: Per-class precision and recall

Class	SVM precision	SVM recall	NB precision	NB recall
Roads	1.000	0.944	0.971	0.917
Water	0.914	0.889	0.868	0.917
Sanitation	0.970	0.889	0.914	0.889
Electricity	0.833	0.972	0.846	0.917
Health	0.917	0.917	0.971	0.917

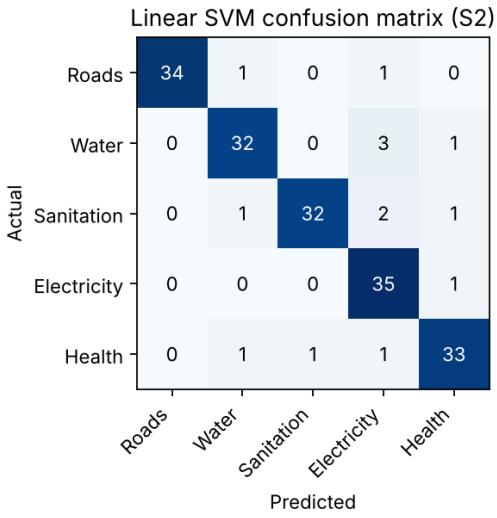


Figure 1: Confusion matrix of the Linear SVM (setting S2)

Hand check (SVM, class Electricity). The Electricity column of Table 6 is 1 + 3 + 2 + 35 + 1 = 42 predictions, of which 35 are correct, so precision = 35/42 = 0.833; the Electricity row has 36 complaints and 35 are found, so recall = 35/36 = 0.972. Both agree with Table 8. Seven of the 14 SVM errors are complaints wrongly sent **into Electricity**, which has the strongest single keywords ("power", "wire", "transformer").

4. k-means clustering of the unlabelled complaints

All 600 complaints were clustered without labels with k-means, k = 5, 10 random starts, seed 42, on the S2 TF–IDF vectors. Table 9 compares the clusters with the true categories.

Table 9: k-means (k = 5, S2 features) – true category (rows) against cluster (columns)

Category	Cl 0	Cl 1	Cl 2	Cl 3	Cl 4
Roads	48	12	48	0	12
Water	13	43	43	3	18
Sanitation	8	10	83	1	18
Electricity	23	10	60	1	26
Health	16	9	56	32	7

Purity = 0.387, Adjusted Rand Index = 0.050. Top terms per cluster: Cl 0 road, main, damag, waterlogg, cave; Cl 1 stand, bus, water, supply; Cl 2 street, market, week, school, temple; Cl 3 breed, mosquito, stagnant, water; Cl 4 bad, very, power, cut, school.

Do the clusters agree with the categories? Only weakly. Cluster 3 is a fairly clean Health cluster (32 of its 37 complaints are Health, and its top terms are mosquito, breed, stagnant) and Cluster 0 is led by Roads (48 of 108 complaints, top terms road, damag, waterlogg), but Cluster 2 (290 complaints) is dominated by the **location and filler words that every category shares** ("school", "market", "week"), so it collects all categories. k-means only sees overall word similarity; it cannot know that "near school" is irrelevant to the topic, whereas the classifiers learned this from the labels. As a test, I removed the shared location and filler words as extra stop-words and re-ran k-means: purity rose from 0.387 to 0.498 and ARI from 0.050 to 0.199, which confirms the explanation, though clustering of such short, template-like texts stays far weaker than classification.

5. Analysis of misclassified documents

Table 10: Examples of misclassified test complaints (Linear SVM, S2)

Complaint text	Actual	Predicted
dead animal on road waste burning in our street	Health	Sanitation
dead animal on road frequent power failure near bus stand again and again	Health	Electricity
public toilet dirty sewage mixing with water near temple kindly act	Sanitation	Water
waste burning loose electric wire on main road kindly act	Sanitation	Electricity
water leakage near school road cave in	Roads	Water

Complaint text	Actual	Predicted
open dumping food stall unhygienic near temple very bad	Sanitation	Health
pipeline burst frequent power failure near bus stand	Water	Electricity
frequent power failure sewage mixing with water since last week	Water	Electricity

Why they were misclassified.

- **Mixed-topic complaints (the main cause).** “dead animal on road waste burning in our street” is labelled Health but also contains sanitation words (“waste”, “burning”); “pipeline burst frequent power failure” contains a Water and an Electricity phrase. A single-label classifier has to pick one, and it picks the category whose words have the larger weight. These errors are not mistakes of the algorithm so much as a sign that the complaint has two topics.
- **Shared words across categories.** “water” appears in Water, Roads (waterlogging), Sanitation (sewage) and Health (stagnant water) complaints, so it is a weak feature; “water leakage near school road cave in” was predicted as Water because “water leakage” outweighs “road cave in”.
- **Short texts.** With only 4–8 words per complaint, one misleading phrase is a large share of the evidence.
- **Class overlap rather than lack of data:** the errors are spread over all classes (Tables 6–7), which fits ambiguity better than a weak class.

Remedy. Multi-label output (assign both topics when the second score is close), phrase (bigram) features, and routing low-confidence complaints to a human reviewer.

TASK 3 – Web Mining: Structure, Usage and Page Importance5 Marks

The Corporation's portal has five pages with links: H → S, C; S → C; C → H; N → H, C, A; A → N (H = Home, S = Services, C = Complaints, N = News, A = Archive).

- Distinguish web content mining, web structure mining and web usage mining, and give one use of each in this scenario
- Compute PageRank for two iterations with damping factor d = 0.85, starting with equal ranks of 0.2, showing every step. Which page is the most important and why?
- Explain the idea of hubs and authorities and which pages might be hubs
- Explain what web page layout structure means (for example the DOM tree and block segmentation) and why navigation menus and advertisements should be separated from the main content before mining

Evidence Expected

Web mining categories with examples, PageRank iterations, hub/authority explanation and a layout-structure discussion.

Ans:

1. Three kinds of web mining

Type	What is mined	Use in Metro City
Web content mining	The content of pages: text, titles, images (text mining, classification).	Classify portal and news pages into Roads, Water, Sanitation, Electricity, Health or Other (Task 4); group complaint text by topic.
Web structure mining	The hyperlink graph between pages (PageRank, HITS).	Find which portal pages are most important or authoritative, so the key service pages are easy to reach and are crawled first.
Web usage mining	Server logs and click-streams: sessions, paths, time on page.	Find that citizens go Home → Services → Complaints but often leave before submitting, and redesign the form; detect peak hours.

2. PageRank, two iterations (d = 0.85, N = 5)

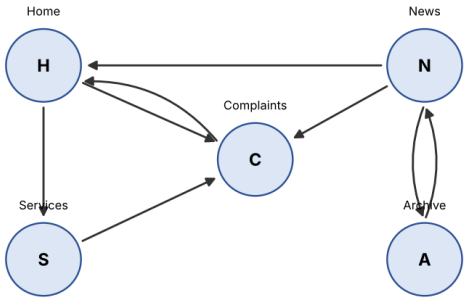


Figure 2: Link graph of the portal

Out-links: H → S, C (2 links); S → C (1); C → H (1); N → H, C, A (3); A → N (1). In-links: H ← C, N; S ← H; C ← H, S, N; N ← A; A ← N. Formula: PR(p) = (1 − d)/N + d × Σ PR(q)/L(q), where (1 − d)/N = 0.15/5 = **0.03**. Start: PR = 0.2 for every page.

Iteration 1 (all ranks 0.2)

PR(H) = 0.03 + 0.85 × (PR(C)/1 + PR(N)/3) = 0.03 + 0.85 × (0.2 + 0.0667) = 0.03 + 0.2267 = 0.2567

PR(S) = 0.03 + 0.85 × (PR(H)/2) = 0.03 + 0.85 × 0.1 = 0.1150

PR(C) = 0.03 + 0.85 × (PR(H)/2 + PR(S)/1 + PR(N)/3) = 0.03 + 0.85 × (0.1 + 0.2 + 0.0667) = 0.03 + 0.3117 = 0.3417

PR(N) = 0.03 + 0.85 × (PR(A)/1) = 0.03 + 0.85 × 0.2 = 0.2000

PR(A) = 0.03 + 0.85 × (PR(N)/3) = 0.03 + 0.85 × 0.0667 = 0.0867

Check: 0.2567 + 0.1150 + 0.3417 + 0.2000 + 0.0867 = 1.0001 ≈ 1.

Iteration 2 (using the iteration-1 values)

PR(H) = 0.03 + 0.85 × (0.3417/1 + 0.2000/3) = 0.03 + 0.85 × 0.4084 = 0.3771

PR(S) = 0.03 + 0.85 × (0.2567/2) = 0.03 + 0.85 × 0.1283 = 0.1391

PR(C) = 0.03 + 0.85 × (0.2567/2 + 0.1150/1 + 0.2000/3) = 0.03 + 0.85 × 0.3100 = 0.2935

PR(N) = 0.03 + 0.85 × (0.0867/1) = 0.1037

PR(A) = 0.03 + 0.85 × (0.2000/3) = 0.03 + 0.0567 = 0.0867

Table 11: PageRank values

Page	Start	Iteration 1	Iteration 2	Rank after iteration 2
H – Home	0.2000	0.2567	0.3771	1
C – Complaints	0.2000	0.3417	0.2935	2
S – Services	0.2000	0.1150	0.1391	3
N – News	0.2000	0.2000	0.1037	4
A – Archive	0.2000	0.0867	0.0867	5

Most important page: Home (H) after two iterations (0.377). Complaints (C) led after iteration 1, but C passes **all** of its rank to H through its only out-link, and H is also linked from N, so H overtakes C. Both pages sit on a H ⇄ C cycle that keeps feeding rank back. Running the same formula to convergence (a check with a short program, not the hand calculation) gives H = 0.348 and C = 0.350, so after the ranks have settled **H and C are nearly tied at the top**, while N (0.073) and A (0.051) are at the bottom because only the Archive links to News and the Archive receives just a third of News's rank. Practical meaning: Home and Complaints are the pages the portal's link structure treats as the centre.

3. Hubs and authorities

In the HITS idea, every page has two scores. An **authority** is a page that many good hubs link to (it holds the content people need); a **hub** is a page that links to many good authorities (it is a good directory). The two scores reinforce each other and are found iteratively. In this portal, **N (News)** links to three pages (H, C, A) and **H (Home)** links to S and C, so these are the likely hubs; **C (Complaints)** has three in-links (from H, S and N), so it is the strongest authority, and H is a second authority because C and N both point to it. S has one in-link and only one out-link, so it is neither strong hub nor authority.

4. Page layout structure and why boilerplate is separated

A web page is not just a bag of words: its HTML forms a **DOM tree** (html → body → header, nav, div, article, aside, footer → paragraphs, links, images). **Layout structure** is the way this tree is displayed as blocks: header, navigation menu, main article, sidebar, advertisements, footer. **Block segmentation** (for example the VIPS algorithm, or simple heuristics on tag type, text density and link density) cuts the page into such visual blocks so that each block can be labelled as main content or boilerplate.

Navigation menus and advertisements must be removed before mining because:

- **They repeat on every page.** "Home Services Complaints News Archive" appears on all pages, so these words get a high term frequency and make all pages look alike for classification and clustering; IDF only partly corrects this.
- **Advertisements and sidebars add off-topic words** (a mobile-phone advertisement on a water-leakage page can push it towards "Other").
- **They distort the link structure.** Site-wide menu links are not real endorsements; if counted, every page appears to link to every other page and PageRank loses its meaning. Link-density (links ÷ words) is the standard signal: blocks with very high link density are menus.
- **They waste storage and time** and cause near-duplicate pages to look different.

TASK 4 – Automatic Classification of Web Documents5 Marks

The Corporation wants to classify web pages (portal pages and linked news pages) into the categories Roads, Water, Sanitation, Electricity, Health and Other.

- Design the complete pipeline: collection → cleaning (removal of tags, scripts and boilerplate) → feature extraction → training → evaluation → deployment
- Identify the features that are specific to web pages (for example title, headings, URL tokens, anchor text, hyperlinks from neighbouring pages) and explain how each would help
- Explain the challenges of web data (noise, multi-topic pages, dynamic content, class imbalance) and one remedy for each
- State how you would evaluate the classifier and detect when it needs retraining

Evidence Expected

A labelled pipeline diagram, a feature table with justification and an evaluation plan.

Ans:

1. Pipeline

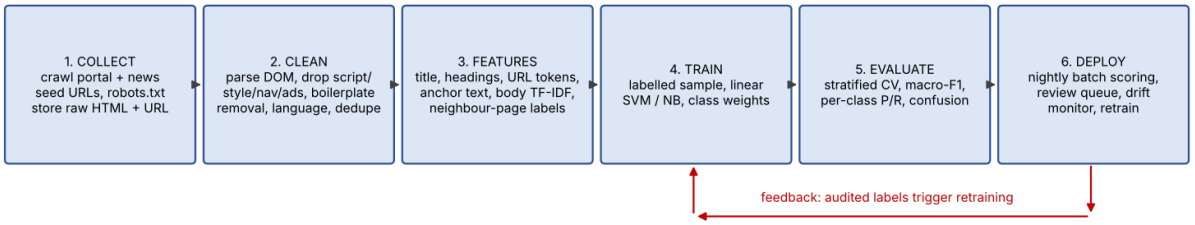


Figure 3: Pipeline for classifying portal and news pages

Stage	What happens and why
1. Collection	A focused crawler starts from the portal sitemap and the news sites' RSS feeds, obeys robots.txt, and stores the raw HTML with URL, fetch time and HTTP status. About 200 pages are labelled by hand at the start and more through the review queue.
2. Cleaning	Parse the HTML into a DOM tree; delete script, style, comment and tag markup; remove navigation, advertisements and footers by block segmentation and link density (Task 3); detect the language; remove exact and near-duplicates (shingling / simhash).
3. Feature extraction	Separate fields for title, headings, URL tokens, anchor text, body text (TF–IDF with stop-words and stemming), plus neighbour-page features (see Section 2). Fields are weighted, for example title ×3.
4. Training	Stratified labelled set (for example 3,000 pages). Linear SVM as the main model and Naïve Bayes as a fast baseline; class weights for the minority classes; hyper-parameters by cross-validation.
5. Evaluation	Stratified 5-fold cross-validation and a time-ordered hold-out set; per-class precision, recall, F1, macro-F1 and the confusion matrix (Section 4).
6. Deployment	Nightly batch scoring inside the ETL job; the label and probability are stored in the warehouse; predictions below 0.6 confidence go to a review queue; reviewed labels return to the training set.

2. Features specific to web pages

Feature	Why it helps
Title	Short, written to summarise the page ("Pothole repair schedule – Roads Department"); a few words carry the topic, so it gets the highest weight.
Headings (h1–h3)	Show the structure of the page and the topics of its sections; useful for pages with several topics.
URL tokens	/roads/pothole-repair or /water/supply-schedule often name the department; works even when the page text is poor or not yet rendered.
Anchor text of in-links	What other pages say about the page ("report a garbage problem") – an outside description that is often more concise than the page itself.

Feature	Why it helps
Hyperlinks from / to neighbouring pages	Pages linked from a Roads page are often Roads pages (topic locality). The labels or predicted classes of neighbours become extra features, which helps for pages with little text such as image galleries.
Meta keywords, image alt text, breadcrumb	Cheap, human-written hints of the topic and the position in the site hierarchy (Home > Utilities > Water).
Body text TF–IDF	The main evidence after boilerplate removal; carries the specific vocabulary (“transformer”, “dengue”).

3. Challenges of web data and remedies

Challenge	Problem	Remedy
Noise	Menus, advertisements, comments, spelling errors and scripts add irrelevant words.	Boilerplate removal by block segmentation and link density; give extra weight to title and headings; remove near-duplicates.
Multi-topic pages	A news page on monsoon may cover waterlogged roads, power cuts and dengue.	Segment into blocks and classify each block, or use multi-label output and keep the top-2 classes above a threshold.
Dynamic content	Text loaded by JavaScript is missing in the raw HTML; pages change daily.	Render with a headless browser, or use the site’s API or feed; store a dated snapshot and re-crawl by change frequency.
Class imbalance	“Other” may be 60% of pages and Health only 3%, so a classifier can look accurate while missing Health.	Class weights or oversampling of small classes; stratified sampling; judge by macro-F1 and per-class recall, not accuracy.

4. Evaluation and retraining

- **Offline evaluation:** 5-fold stratified cross-validation; confusion matrix; precision, recall and F1 for each class; **macro-F1 as the headline number** (target ≥ 0.85, and recall ≥ 0.80 for every class, including Health). A time-ordered hold-out set (train on older pages, test on the newest month) is more honest than random splits because the web changes.
- **Online audit:** each week 100 randomly chosen predicted pages (and all low-confidence ones) are labelled by staff; the audit accuracy is plotted over time.
- **Signs that retraining is needed:** (1) audit macro-F1 falls by more than 5 points from the validation value; (2) the share of low-confidence predictions or of “Other” rises noticeably; (3) the **out-of-vocabulary rate** grows (new words such as the name of a new disease or scheme); (4) the portal is redesigned (changed URL patterns and templates); (5) the class mix changes strongly. In addition, retrain every month with the reviewed labels, and always keep the previous model to roll back.

TASK 5 – Spatial Data and Spatial Association Patterns6 Marks

- Describe the main spatial data types (point, line, polygon, raster) and the main spatial relationships (topological, distance, direction) with an example of each from the city data
- Explain spatial autocorrelation and why standard statistical methods that assume independent observations can mislead on spatial data

The map shows instances of three feature types: A (accident spots), B (bars), C (cross-road junctions without lights). There are 4 instances of A (A1–A4), 5 of B (B1–B5) and 3 of C (C1–C3). Neighbouring pairs: A–B: (A1,B1), (A1,B2), (A2,B2), (A3,B4); A–C: (A1,C1), (A2,C1), (A3,C2); B–C: (B1,C1), (B2,C1).

- For each pair, compute the participation ratio of each feature and the participation index
- With a minimum prevalence threshold of 0.5, identify the prevalent size-2 co-locations
- The only instances forming the triple are (A1,B1,C1), (A1,B2,C1) and (A2,B2,C1). Compute the participation index of {A, B, C} and decide whether it is prevalent
- Explain why a co-location is not the same as a market-basket association rule

Evidence Expected

Spatial data and relationship examples, participation ratio/index calculations and a correct decision on prevalent co-locations.

Ans:

1. Spatial data types and relationships

Table 12: Spatial data types with examples from Metro City

Type	Description	Example from the city data
Point	A single coordinate pair (x, y)	Each GPS-recorded road accident or incident; a bar; a cross-road junction.
Line	An ordered chain of points (polyline)	A road centre-line, a water pipeline, a bus route.
Polygon	A closed area bounded by lines	Ward and zone boundaries, a flood-prone area, the grounds of a school.
Raster	A grid of cells, each with a value	A satellite image, a ground-elevation grid, a heat map of complaint density per 250 m cell.

Table 13: Spatial relationships with examples

Relationship	Meaning	Example
Topological	Relations that do not change under stretching or rotation: equals, disjoint, touches, crosses, within, contains, overlaps.	An accident point is **within** Ward 12; the Mount Road line **crosses** the Adyar river polygon; Ward 12 **touches** Ward 13.
Distance	Metric distance (Euclidean, or along the road network).	Hospital is **within 500 m** of the accident spot; the nearest fire station is 3.2 km away by road.
Direction	Orientation of one object relative to another (north of, east of, above, below).	The flyover is **north of** the bus stand; the junction lies **south-east** of the school.

2. Spatial autocorrelation

Spatial autocorrelation is the correlation of a variable with itself across space: values at nearby locations are more similar than values far apart (Tobler's first law of geography: "everything is related to everything else, but near things are more related than distant things"). It is measured by indices such as **Moran's I** (positive = similar values cluster, negative = neighbours differ, near 0 = random) and Geary's C. Example: potholes appear in clusters on the same worn stretch of road after the same monsoon.

Why standard statistics can mislead. Classical tests and regressions assume that observations are **independent**. With positive spatial autocorrelation, 1,000 neighbouring observations contain much less information than 1,000 independent ones (the effective sample size is smaller), so (1) standard errors are **too small** and confidence intervals too narrow, (2) p-values are too optimistic and chance patterns are called "significant", (3) regression residuals are correlated, which makes the coefficient estimates inefficient and sometimes biased, and (4) a random train–test split leaks information because test points have training neighbours, so accuracy looks better than it is. Remedies: spatial regression (spatial lag or error models), geographically weighted regression, and block-wise (spatial) cross-validation.

3. Participation ratios and participation index

For a feature f in a pair (or set) of features, **participation ratio** PR(f) = (number of distinct instances of f that take part in at least one neighbouring pair/row instance) / (total number of instances of f). The **participation index** is PI = min over the features of PR. Totals: A = 4, B = 5, C = 3.

Table 14: Size-2 co-locations

Pair	Participating instances	Participation ratios	PI = min	Prevalent (≥ 0.5)?
{A, B}	A: A1, A2, A3 (A4 not); B: B1, B2, B4 (B3, B5 not)	PR(A) = 3/4 = 0.75; PR(B) = 3/5 = 0.60	**0.60**	Yes
{A, C}	A: A1, A2, A3; C: C1, C2 (C3 not)	PR(A) = 3/4 = 0.75; PR(C) = 2/3 = 0.667	**0.667**	Yes
{B, C}	B: B1, B2; C: C1 only	PR(B) = 2/5 = 0.40; PR(C) = 1/3 = 0.333	**0.333**	No

Prevalent size-2 co-locations at min_prev = 0.5: {A, B} (PI = 0.60) and {A, C} (PI = 0.667). {B, C} is not prevalent (PI = 0.333): only one of the three junctions has a bar next to it, and two of five bars are next to it.

4. The triple {A, B, C}

Row instances: (A1,B1,C1), (A1,B2,C1), (A2,B2,C1). Distinct participating instances: A = {A1, A2}, B = {B1, B2}, C = {C1}.

PR(A) = 2/4 = 0.50 PR(B) = 2/5 = 0.40 PR(C) = 1/3 = 0.333

PI({A,B,C}) = min(0.50, 0.40, 0.333) = 0.333 < 0.5 → NOT prevalent

This could be predicted without counting: the participation index is **anti-monotone** (adding a feature can only keep or lower PI), and {B, C} already has PI = 0.333 < 0.5, so no superset of {B, C} can be prevalent. Candidate pruning on this property is what makes co-location mining efficient. The result is also sensible: junction C1 is the only junction involved in the triple.

5. Co-location is not a market-basket association rule

- **There are no transactions.** Market-basket mining counts baskets that contain items; space is continuous and has no natural basket. The neighbour relation (for example "within 200 m") creates overlapping groups, and one instance (C1) can belong to many of them.
- **Counting.** Support = (baskets containing the items) / (all baskets). In space, counting row instances would count the same accident several times (A1 appears in two rows of the triple), so frequency is biased. The participation ratio/index counts **distinct instances** of each feature instead and so is fair to common and rare features.
- **The unit of analysis is the feature type with many instances, not an item.** Rules such as "bar → accident" are expressed by conditional participation (how many bars have an accident nearby), not by confidence = P(B | A) over baskets.
- **Spatial effects:** the instances are autocorrelated and the result depends on the distance threshold and the map extent, so the same data can give different co-locations; market-basket items are assumed to be independent events.

TASK 6 – Spatial Clustering of Incident Locations5 Marks

- Use real or synthetic GPS incident data (at least 1,000 points) that contains dense hot-spots of different shapes and scattered noise points. Document the source.
- Run k-means and DBSCAN (and discuss, without implementing, how a medoid-based method such as CLARANS would differ)
- Select the DBSCAN parameters (ε and MinPts) using a k-distance graph and show the effect of changing ε
- Plot the clusters, and identify the points treated as noise
- Explain which method better represents the hot-spots and why, based on how each method defines a cluster

Evidence Expected

Plots, parameter-selection evidence and an interpreted comparison of the methods.

Ans:

1. Data and its source

Source: synthetic, generated by `spatial.py` (Appendix B), NumPy random generator with seed 42; data file `incidents.csv` (columns lat, lon, true_group). The 1,500 points lie in a 10 km × 10 km window near 12.95° N, 80.15° E (kilometre coordinates are converted to latitude/longitude with 1° latitude = 111 km). They contain four hot-spots of different shapes and 200 uniformly scattered noise points:

Hot-spot	Shape	Points	Generation
A	Large round blob	400	Gaussian, σ = 0.45 km, centre (2.5, 7.0)
B	Long thin strip along a road	350	Points on a line from (1, 2) to (8, 3.2), spread 0.10 km
C	Crescent / arc	350	Half-circle of radius 1.5 km, spread 0.09 km
D	Small, very dense blob	200	Gaussian, σ = 0.22 km, centre (8.3, 2.0)
Noise	Random scatter	200	Uniform over the whole window

Because I generated the data, the true group of every point is known, so the Adjusted Rand Index (ARI, 1 = perfect agreement with the truth) can be used to judge the methods; with real data one would use expert review and silhouette-type measures.

2. DBSCAN parameters from the k-distance graph

MinPts = 8. The minimum is dimension + 1 = 3, and the usual rule is 2 × dimension = 4; with 1,500 points and 13% noise a larger value avoids connecting noise points through chance chains, and 8 is also the smallest group I would call a "hot-spot" of incidents. **ε:** for every point I computed the distance to its 8th nearest neighbour and sorted these values (Figure 4). Points inside a dense hot-spot have small values; noise points have large values; the "knee" where the curve turns sharply upward separates the two. The knee (point of maximum distance from the straight chord) is at **ε = 0.243 km**, so I chose **ε = 0.25 km**.

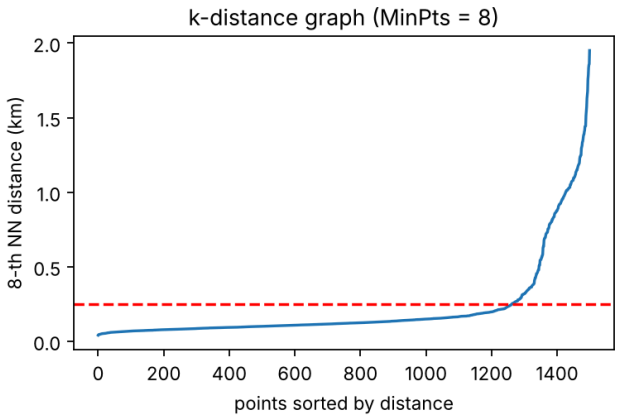


Figure 4: k-distance graph (k = 8); the dashed line is ε = 0.25 km

(Percentiles of the 8-NN distance: 50% = 0.12 km, 80% = 0.20 km, 85% = 0.27 km, 90% = 0.55 km, which shows the sudden rise beyond about 85% of the points, close to the 87% that are not noise.)

3. Effect of changing ε (MinPts = 8)

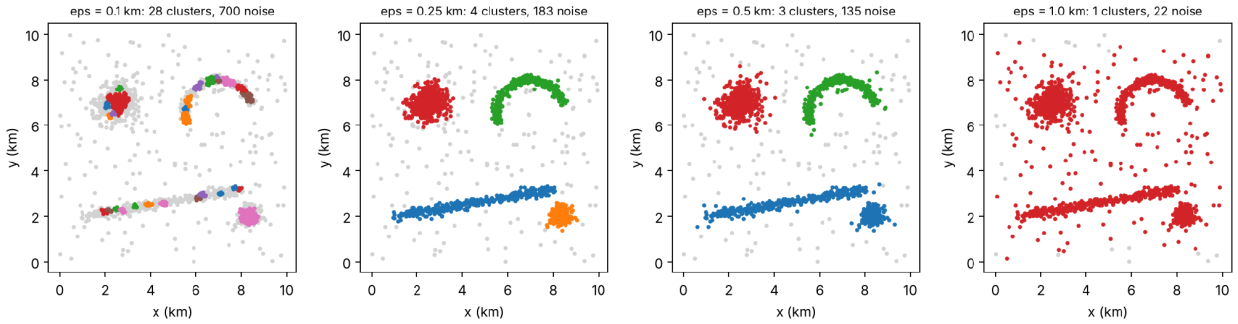


Figure 5: DBSCAN with ε = 0.10, 0.25, 0.50 and 1.00 km (grey = noise)

Table 15: Effect of ε

ε (km)	Clusters	Noise points	True noise recovered	ARI vs truth	Observation
0.10	28	700	94%	0.184	Too small: hot-spots broken into 28 fragments, 47% of all points called noise
0.25	4	183	82%	0.925	**Four clusters = four true hot-spots**
0.50	3	135	68%	0.745	Strip B and dense blob D merge into one cluster; more noise absorbed
1.00	1	22	11%	0.007	Too large: everything joins into one cluster

4. k-means, DBSCAN, and the noise points

k-means used k = 4 (the true number of hot-spots, which is a favour to k-means, because in practice k is unknown), 10 random starts, seed 42.

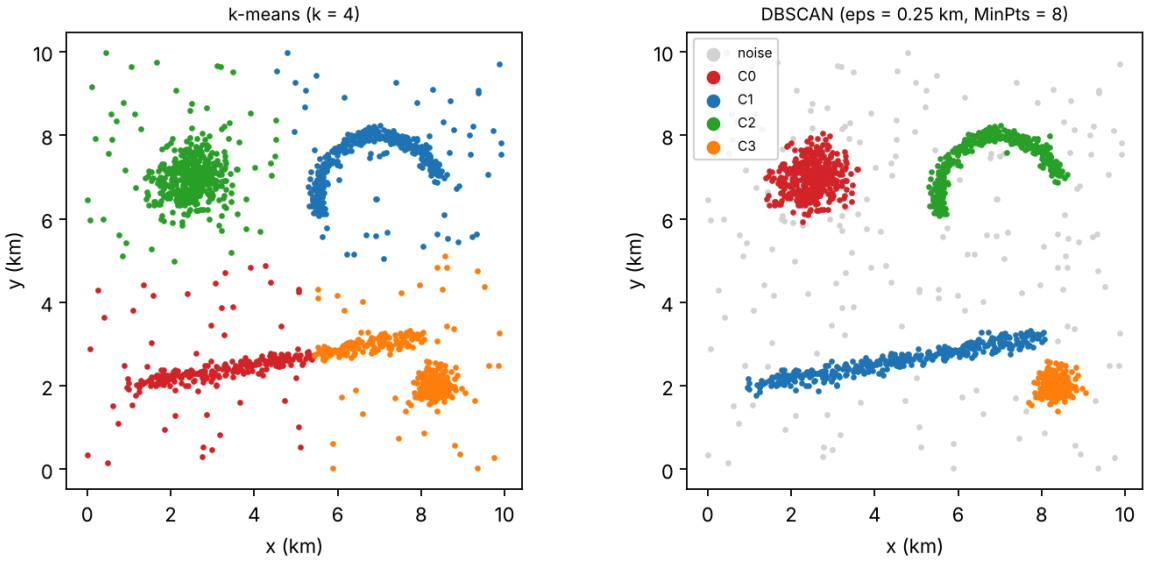


Figure 6: k-means (k = 4) and DBSCAN (ε = 0.25 km, MinPts = 8)

Method	Clusters found	Noise identified	ARI vs truth	What happens
k-means (k = 4)	4	0 (every point is forced into a cluster)	0.665 (0.827 on non-noise points only)	The long strip is **cut into two**; the small dense blob D is merged with half of the strip; the round blob and crescent absorb the surrounding noise and the cluster centres are pulled away from the hot-spots.
DBSCAN (ε = 0.25, MinPts = 8)	4	183 points shown in grey	0.925	Round blob, strip, crescent and small blob are each recovered as one cluster with its true shape; 164 of the 200 true noise points (82%) are labelled noise; the other 19 noise-labelled points are thin edge points of the hot-spots.

5. CLARANS (discussed, not implemented)

CLARANS is a **medoid-based** method: it also needs k in advance, but each cluster is represented by a **medoid, an actual data point**, instead of a mean. It searches randomly among neighbouring sets of medoids (a graph in which two solutions are neighbours if they differ in one medoid), examining at most "maxneighbor" random swaps and restarting "numlocal" times, and keeps the set of medoids with the lowest total distance. Compared with k-means it (1) is **less sensitive to noise and outliers**, because a medoid cannot be dragged away by a few distant points and the cost is a sum of distances, not squared distances; (2) can use any dissimilarity, for example road-network distance; and (3) costs more time (about $O(n^2)$ per search, acceptable for 1,500 points but heavy for millions). However, it still assigns each point to the nearest medoid, so clusters are compact, roughly convex regions; it would still **cut the road strip and crescent** and would still not label noise. It would be better than k-means on this data but worse than DBSCAN for the hot-spot shapes.

6. Which method represents the hot-spots better, and why

DBSCAN is clearly better (ARI 0.925 against 0.665). The difference follows from how each method **defines a cluster**. k-means (and CLARANS) define a cluster as the set of points closest to one centre, so every cluster is a convex, roughly round cell of a Voronoi partition; a long road-side strip or a crescent cannot be one such cell, and the number of clusters must be given. Noise has no place in the definition, so scattered points are pushed into the nearest cluster and move its centre. DBSCAN defines a cluster as a maximal set of **density-connected** points: a core point has at least MinPts points within ε, and clusters grow by chaining through core points, so any shape can be followed and points that are in no dense neighbourhood are noise. For incident hot-spots, which are exactly dense places along roads and junctions, this is the natural definition. **Weaknesses of DBSCAN** that I observed: results are sensitive to ε (Table 15: 28 clusters at 0.10, 1 cluster at 1.00), and one global ε cannot suit hot-spots of very different density (the dense blob D and the strip merge at $\varepsilon = 0.5$). Remedies are OPTICS or HDBSCAN.

TASK 7 – Social Impacts and Privacy4 Marks

The following extract is to be released to researchers: (Age, PIN code, Complaint type) = (23, 600041, Garbage), (27, 600042, Garbage), (24, 600041, Water), (52, 600087, Roads), (58, 600088, Roads), (55, 600087, Water).

- Treating Age and PIN code as quasi-identifiers, determine the k-anonymity of the table as given
- Generalise the quasi-identifiers (for example age ranges and PIN-code prefixes) to achieve 2-anonymity, and state what information is lost
- Identify three social risks of mining the platform's data (for example profiling, bias against neighbourhoods, misuse) and a safeguard for each

Evidence Expected
k-anonymity analysis, the anonymised table and a risk–safeguard table.

Ans:

1. k-anonymity of the table as given

A table is **k-anonymous** if every combination of quasi-identifier values (an equivalence class) occurs in at least k records, so that each person is hidden among at least k people. I group the rows by (Age, PIN code):

Row	Age	PIN code	Complaint	Equivalence class (Age, PIN)	Class size
1	23	600041	Garbage	(23, 600041)	1
2	27	600042	Garbage	(27, 600042)	1
3	24	600041	Water	(24, 600041)	1
4	52	600087	Roads	(52, 600087)	1
5	58	600088	Roads	(58, 600088)	1
6	55	600087	Water	(55, 600087)	1

Every combination appears only once, so the smallest class has size 1: **the table is 1-anonymous (k = 1), i.e. not anonymous at all.** Anyone who knows that a neighbour is 24 and lives in 600041 (for example from an electoral roll or a social-media profile) can find that person's record and learn that the complaint was about water.

2. Generalisation to 2-anonymity

Generalise Age into 10-year bands and the PIN code to its first five digits (the last digit is replaced by *):

Age (generalised)	PIN code (generalised)	Complaint type	Class size
20–29	60004*	Garbage	3
20–29	60004*	Garbage	3
20–29	60004*	Water	3
50–59	60008*	Roads	3
50–59	60008*	Roads	3
50–59	60008*	Water	3

There are now two equivalence classes, (20–29, 60004*) and (50–59, 60008*), each with 3 records, so the smallest class is 3 and the table is **3-anonymous, which also satisfies the required 2-anonymity.** (Generalising only one attribute is not enough: with age bands but exact PIN codes, the record (27, 600042) would still be alone in its class (20–29, 600042).)

Information lost: (1) exact ages: an analyst can no longer separate a 23-year-old from a 29-year-old, so age-specific patterns and averages are blurred by up to ±5 years; (2) exact PIN codes: 600041 and 600042 are merged, so the street- and neighbourhood-level comparison is lost and only the larger area is known; (3) because the groups are small, statistics by area cannot be computed more finely than the 60004* area. The complaint type is kept unchanged. **Remaining risk:** in the class (20–29, 60004*) two of the three complaints are Garbage, so an attacker who knows a person is in that class can guess "Garbage" with 67% certainty. k-anonymity does not protect the sensitive attribute; l-diversity (each class has at least l different complaint types; here l = 2) or t-closeness gives further protection.

3. Social risks of mining the platform's data

Risk	How it can harm	Safeguard
1. Profiling and surveillance of individuals	Linking complaints, app and portal clicks with location can reveal who complains, when they are at home, and how they behave; repeat complainants may be flagged as "troublemakers".	Data minimisation and pseudonymous IDs from ETL; no personal identifiers or exact home locations in the analytic platform; role-based access with audit logs; purpose limitation (the data may be used only for service improvement).
2. Bias against neighbourhoods	Wards with fewer smartphones or less literate residents file fewer complaints, so models and hot-spot maps under-serve them; areas labelled "high crime / high complaint" can be stigmatised, and decisions based on the model can create a feedback loop.	Normalise by population and by reporting rate; add non-complaint data (sensor, field inspection); fairness audit of model errors by ward and by language; human review before resource decisions; publish methods.
3. Misuse and function creep, including re-identification	Data collected for services is used for policing, political targeting or sold; released tables can be linked to other data (Task 7 part 1) to identify people.	Legal data-sharing agreements and a data-protection officer; k-anonymity ($k \geq 5$) with l-diversity and suppression of small cells before any release; differential-privacy noise for public statistics; retention limits and deletion; breach monitoring.

TASK 8 – Design the Integrated System6 Marks

Design the integrated warehousing and mining platform for the Commissioner. Your design must include: a labelled architecture showing data sources, ETL (including how unstructured text and spatial data are loaded), the warehouse, the mining modules and the reporting layer; a dimensional design (fact and dimensions) for service requests, including a location dimension and a time dimension; a mapping of each requirement of the Commissioner to the technique you would use and the reason; the privacy controls built into the design; how you will evaluate whether each module is working. A diagram without reasoning will not receive full credit.

Evidence Expected

Architecture diagram, dimensional design, requirement-to-technique mapping, privacy controls and an evaluation plan.

Ans:

1. Architecture

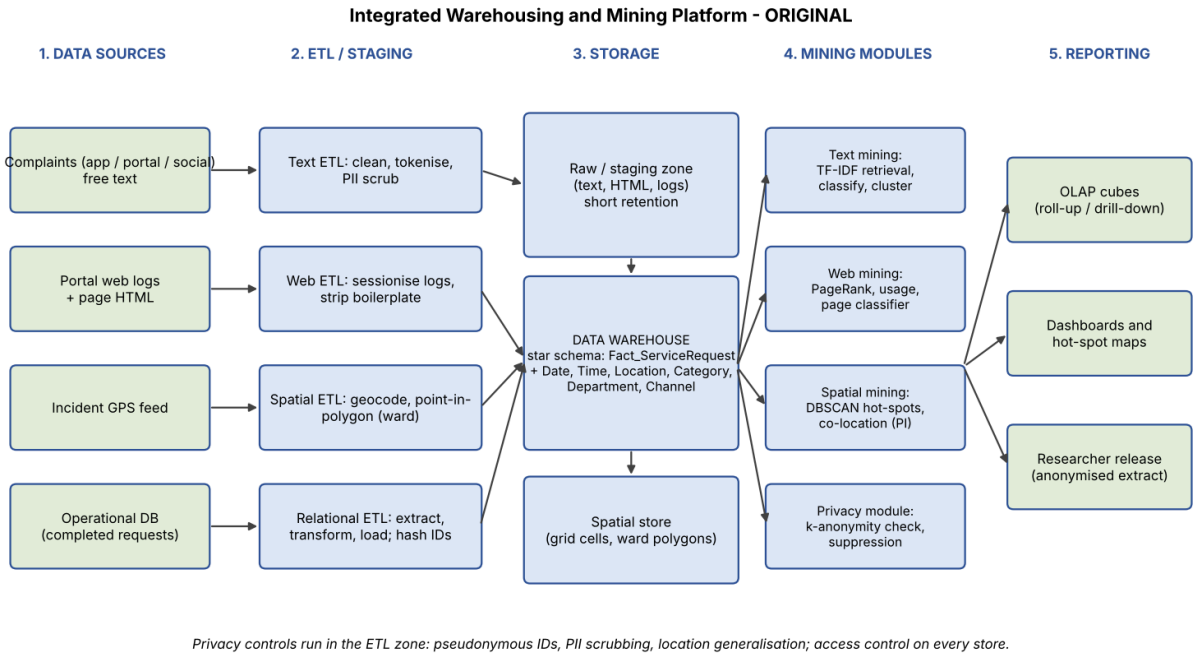


Figure 7: Original architecture (blue = components, green = sources and outputs)

Reasoning. Data flow is one-directional so that privacy can be enforced once, at the ETL boundary, before anything reaches the warehouse. Metro City produces about 5,000 complaints a day (about one every 17 seconds), which a nightly batch ETL with a short intraday refresh handles easily; a streaming engine is not needed. The four sources differ in kind, so each has its own ETL path:

- **Complaint text (unstructured):** the raw text is first kept in a short-retention staging zone. The text ETL cleans it, removes personal details (names, phone numbers, addresses) with pattern and name-list rules, tokenises and stems it, then **text mining turns it into structured attributes**: topic (classifier of Task 2), urgency score (keyword and classifier score), language and cluster id. These attributes are loaded as columns of the fact table and the Category dimension; the cleaned text itself stays in the staging zone for retrieval (TF–IDF index of Task 1).
- **Portal logs and pages (semi-structured):** sessionised by user token and 30-minute inactivity; IP addresses are hashed; page HTML is cleaned of boilerplate before page classification; sessions and page labels are loaded into a web-usage mart.
- **Incident GPS (spatial):** the spatial ETL validates coordinates, joins each point to its ward polygon (point-in-polygon) and loads the result in the spatial store and the Location dimension; the hot-spot module reads the spatial store.
- **Operational database:** classic extract–transform–load of completed requests with surrogate keys and slowly changing dimensions for department and ward names; the citizen ID is replaced by a salted hash.

2. Dimensional design

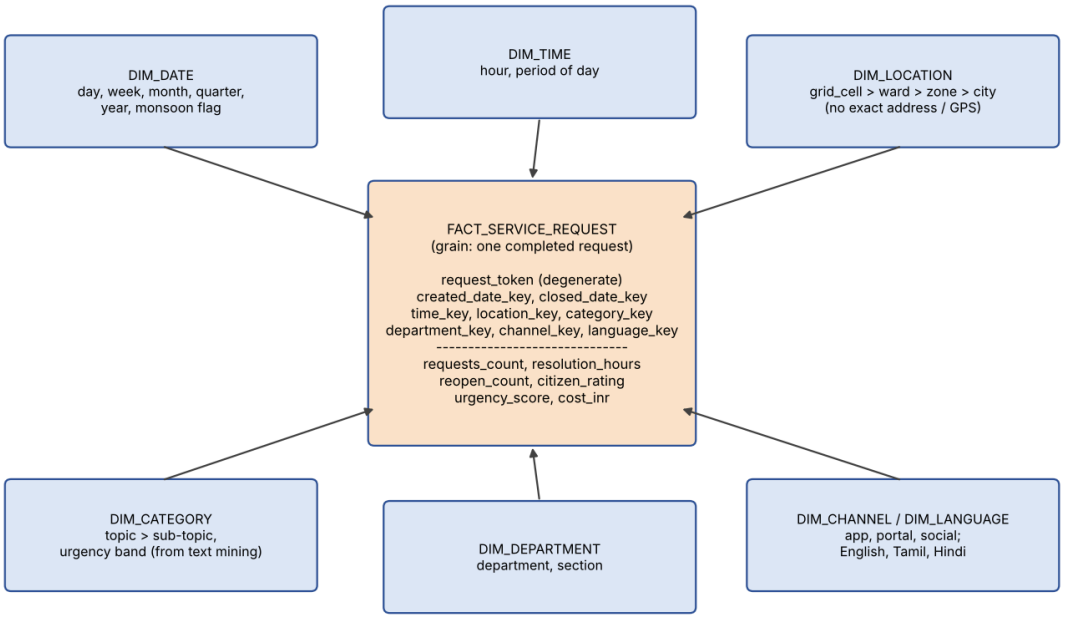


Figure 8: Star schema for service requests

Table 16: Fact and dimension tables (original design)

Table	Key attributes	Notes
Fact_ServiceRequest	Grain: one completed service request. Keys: created_date_key, closed_date_key, time_key, location_key, category_key, department_key, channel_key, language_key; degenerate request_token. Measures: requests_count (1), resolution_hours, reopen_count, citizen_rating, urgency_score, cost_inr.	Additive: requests_count, cost_inr. Semi-additive/averaged: resolution_hours, rating, urgency. Two date keys = role-playing Date dimension.
Dim_Date	date_key, date, day, week, month, quarter, year, weekday/weekend, public holiday, monsoon season flag.	Hierarchy day → week → month → quarter → year; seasonal reports (monsoon waterlogging).
Dim_Time	time_key, hour, period (night, morning peak, day, evening peak).	Kept separate from Dim_Date to avoid 365 × 1,440 rows; supports "time of day" analysis.
Dim_Location	location_key, street segment, ward, zone, city; ward polygon id; centroid (street segment level).	Hierarchy street segment → ward → zone → city for roll-up and drill-down; **no citizen name, house number or exact home address**.
Dim_Category	category_key, topic (Roads, Water, Sanitation, Electricity, Health, Other), sub-topic, urgency band.	Filled from text mining; type 2 slowly changing when the category scheme changes.
Dim_Department, Dim_Channel	Department and section; channel (app, portal, social media).	Standard lookup dimensions.

3. Requirement-to-technique mapping

Commissioner's requirement	Technique	Reason (evidence in this report)
Retrieve complaints by topic	TF–IDF vector-space retrieval with cosine ranking and query expansion	Ranks by weighted evidence; correct top 2 in Task 1; thesaurus fixes the synonym weakness.
Group complaints by topic and urgency	Supervised classification (SVM, Naïve Bayes) for topic; urgency score; k-means to find unknown themes	SVM reached 0.92 on S2 (Task 2); k-means alone only weakly matched categories (purity 0.39), so it is used for discovery, not for assignment.
Understand portal use	Web usage mining (sessions, paths, drop-off) + PageRank on the link graph	Shows where citizens leave the complaint form; Home and Complaints are the central pages (Task 3).
Classify portal and news pages	Web-page classification pipeline with title, URL and anchor features	Boilerplate-free, field-weighted features and macro-F1 evaluation (Task 4).

Commissioner's requirement	Technique	Reason (evidence in this report)
Geographic hot-spots	DBSCAN with ε from the k-distance graph	Found all four hot-spot shapes, ARI 0.925 against 0.665 for k-means (Task 6).
Relationships between spatial features	Co-location mining with participation index	{A,B} and {A,C} prevalent; handles neighbourhood instead of baskets (Task 5).
Summary reports	Star-schema warehouse and OLAP (roll-up, drill-down, slice)	Fast aggregation by ward, month, topic and channel without scanning text.
No violation of privacy	Pseudonymisation, PII scrubbing, k-anonymity and small-cell suppression	k = 1 for the raw extract, k = 3 after generalisation (Task 7).

4. Privacy controls built into the design

- **At ingestion/ETL:** direct identifiers (name, phone, e-mail, citizen ID) are dropped or replaced by a salted-hash token; the token key is kept in a separate vault; names and phone numbers inside free text are masked.
- **In the data model:** Dim_Location holds street segment and ward only; there is no Dim_Citizen; IP addresses are hashed.
- **Before any release or report:** k-anonymity check ($k \geq 5$) on the quasi-identifiers (age band, PIN prefix, ward) and suppression of cells with fewer than 5 requests; l-diversity for the sensitive complaint type.
- **Access and audit:** role-based and row-level security (a ward officer sees only their ward), encryption at rest and in transit, access logs reviewed monthly.
- **Governance:** purpose limitation, retention limits (raw text deleted after 90 days, tokens after 2 years), consent notice in the app, fairness audit of the models (Task 7, risk 2).

5. Evaluation plan for each module

Module	How I will check that it works	Target
ETL	Row-count and total reconciliation with the source; null, duplicate and key checks; load time; test that no identifier columns exist in the warehouse.	100% reconciliation; load finished before 6 a.m.
Warehouse and reports	Compare 20 report totals with the operational database; query response time; user acceptance by the Commissioner's office.	Zero difference; response < 3 s
Retrieval	Judged queries: precision@k, recall, MAP (Task 1 method).	Top-5 precision ≥ 0.8
Text classification	Stratified cross-validation, macro-F1, confusion matrix; monthly audit of 200 random complaints by staff.	Macro-F1 ≥ 0.85
Web mining	Page classifier: macro-F1 and weekly audit (Task 4); PageRank sanity check against known important pages.	Macro-F1 ≥ 0.85
Hot-spot clustering	Stability across ε and MinPts (Table 15); compare with a benchmark of known hot-spots (ARI); review of the map by field officers.	ARI ≥ 0.8 on benchmark
Co-location mining	Check PI of known pairs; re-run with different distance thresholds; expert review of the reported patterns.	Stable patterns across thresholds
Privacy	k-anonymity ≥ 5 on every release; linkage attack test with a public dataset; review of access logs.	No unique re-identification

TASK 9 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)8 Marks

Changed requirement. A new rule prohibits storing exact residential locations and personal identifiers in the analytical platform. Complaints now arrive in English, Tamil and Hindi, and the daily volume rises to 20,000 complaints. Re-evaluate your design in response to these changes. You may retain, modify or redesign parts of the system, and you must justify the decision.

- A. Impact Analysis (2 marks): identify which modules (ETL, warehouse dimensions, text mining, spatial mining, reports) are affected and which are not.
- B. Evaluation of the Original Design (2 marks): test the original design against the new conditions, for example the effect of multilingual text on your vocabulary and classifier, or of location generalisation on your hot-spot results.
- C. Adaptation or Redesign (2 marks): present the revised design (for example, generalised location hierarchy, anonymisation at ETL, language-detection and per-language or multilingual features). Show the changed parts.
- D. Evidence-Based Defence (2 marks): defend your decision using evidence from your own tasks and state the accuracy or privacy trade-off you accept.

Evidence Expected

A reasoned impact analysis, an evaluation, a revised design and an evidence-based defence.

Ans:

A. Impact Analysis

Three things change: (1) **privacy** – no exact residential location and no personal identifiers may be stored; (2) **language** – English, Tamil and Hindi; (3) **volume** – 5,000 → 20,000 complaints a day (4×; about 7.3 million a year, one every 4.3 seconds on average).

Module	Affected?	Why
ETL	**Yes – major**	New steps are needed: removal of identifiers and PII, location generalisation before loading, language detection, Unicode text handling for Tamil and Hindi scripts. Volume is 4×, but 20,000 records a day is still small for batch ETL (only run-time and storage grow).
Warehouse dimensions	**Yes – Dim_Location, Dim_Language; fact unchanged**	Dim_Location loses street-segment/centroid detail and gets a grid-cell → ward → zone → city hierarchy; a new Dim_Language is added; the degenerate request_token is replaced by a one-way token. The fact table grain, measures and the Date, Time, Department and Channel dimensions are **not affected**.
Text mining	**Yes – major**	Vocabulary, stop-word lists, stemmers, the classifier and the retrieval index were built for English only; a Tamil or Hindi complaint matches no English term.
Spatial mining	**Yes – moderate**	DBSCAN and co-location mining depend on exact distances; now only cell positions are available, so ε, MinPts and the neighbour relation must be redefined.
Web mining	**Barely**	PageRank, page classification and structure mining use the link graph and public pages, not citizen data. Only the portal log needs IP truncation and pages may be in three languages (a language field is added to the page classifier).
Reports	**Yes – minor**	Location drill-down stops at grid cell/ward; a language slice is added; small cells (< 5) are suppressed; totals and time reports are unchanged.
Privacy module	**Strengthened**	k-anonymity and suppression checks become mandatory gates, no longer optional.

B. Evaluation of the Original Design under the New Conditions

B1. Multilingual text

I tested the text classifier on a small simulation (multilingual.py, Appendix C, seed 42): the same 5 categories and 15 complaint phrases, written in English, Hindi and Tamil in roman letters (illustrative synthetic text, 120 complaints per class per language, 1,800 in total; 70/30 split). I did this in roman script so that the experiment runs with the same Unicode-aware tokeniser; real Tamil and Hindi in native script need the same tokeniser but a language-specific stemmer.

Table 17: Linear SVM accuracy on the multilingual simulation

Model (training data)	Vocabulary	Test: English	Test: Hindi	Test: Tamil	Test: all
Original design: English-only model	39	1.000	0.458	0.268	0.550
Multilingual training, vocabulary capped at 30	30	0.608	0.896	0.500	0.672
Multilingual training, vocabulary capped at 60	60	0.937	1.000	0.742	0.891
Multilingual training, full vocabulary	107	1.000	1.000	1.000	1.000
Hindi-only model	41	0.589	1.000	0.274	0.624
Tamil-only model	36	0.323	0.250	1.000	0.535

Findings. (1) The original English-only model is perfect on English (1.00) but only **0.458 on Hindi and 0.268 on Tamil** (guessing among 5 classes would give 0.20): the correct Hindi and Tamil hits come only from words shared with English ("signal", "street light", "clinic"). Overall 0.55. (2) **Vocabulary triples:** 39 English terms become 107 over the three languages (about 2.7×), so the vocabulary cap that worked for English now removes useful words: capping at 30 gives 0.672 and at 60 gives 0.891, against 1.00 with the full vocabulary. (3) English stop-word lists and stemmers do nothing for Hindi and Tamil (Tamil, an agglutinative language, joins suffixes to words, so one word has many surface forms). (4) Retrieval (Task 1) suffers in the same way: the query "garbage" will not find "kachra" or "kuppai".

B2. Location generalisation and hot-spots

The same 1,500 incident points of Task 6 were snapped to the centre of a square grid cell (the only location that would now be stored) and DBSCAN was re-run, with ε chosen so that neighbouring cells are connected (ε ≈ 1.01 × cell size) and MinPts = 8 (duplicate points in a cell count separately).

Table 18: DBSCAN after generalising the location to a grid

Location stored	ε (km)	Clusters	Noise	ARI vs truth	Agreement with exact result	Distinct locations
Exact GPS (original)	0.25	4	183	0.925	1.00	1,500
250 m grid	0.253	4	162	0.928	0.97	372
500 m grid	0.505	3	113	0.719	0.735	205
1 km grid	1.01	1	0	0	0	93

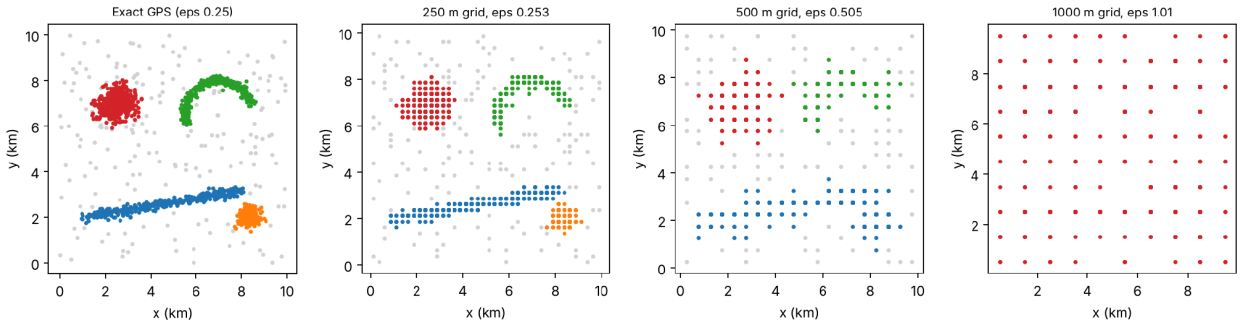


Figure 9: DBSCAN on exact points and on 250 m, 500 m and 1 km grids

Findings. A 250 m grid keeps the four hot-spots (ARI 0.928, 97% agreement with the exact clustering, noise 183 → 162). A 500 m grid merges strip B and blob D into one cluster (3 clusters, ARI 0.719); at 1 km the whole city becomes one cluster (ARI 0.0). Also, the 1,500 points collapse into only 372, 205 and 93 distinct locations, so the data would be much less informative. The original design, which used street-level locations, therefore **cannot be kept as it is**, but it survives if the grid is fine enough.

B3. Volume

At 20,000 complaints a day the TF–IDF matrix, SVM training (seconds on 600 documents in Task 2; minutes on a year of data) and DBSCAN (O(n log n) with a spatial index) still run in minutes; volume is **not** the binding problem.

C. Adaptation: the Revised Design

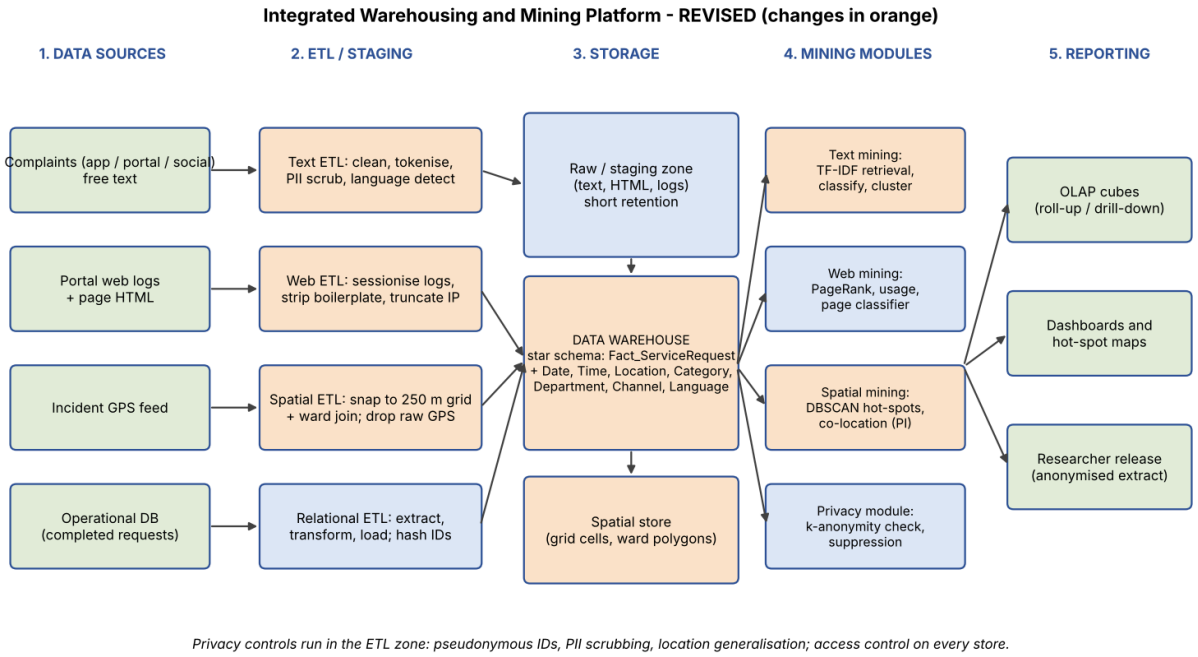


Figure 10: Revised architecture (orange = changed or added parts)

Table 19: What was retained, modified or added

Part	Original	Revised	Decision
Citizen identifier	Salted hash of citizen ID kept in the fact table	No personal identifier stored; only a one-way **request token** per request; the token key and any raw ID stay in the source system	**Modified**
Location (ETL and Dim_Location)	Street segment, ward, centroid; incident GPS in the spatial store	On entry the GPS is **snapped to a 250 m grid cell and a ward id**; the raw coordinates are deleted after the transformation (retained at most 24 hours in a locked staging zone). Hierarchy: grid cell → ward → zone → city. No latitude/longitude, house number or full PIN code (PIN prefix at most).	**Modified**
Language	Not modelled	Language detection at ETL (Unicode script block + short character n-gram model); new **Dim_Language**; language stored on the fact	**Added**
Text preprocessing	English tokeniser, stop-words, suffix stemmer	Unicode-aware tokeniser; **per-language** stop-word lists and light stemmers (a Tamil suffix stripper); PII masking in all three languages; Tamil and Hindi numerals normalised	**Modified**
Text features and classifier	English TF–IDF, vocabulary cap ≈ 500	**One multilingual classifier** trained on all three languages, TF–IDF of words plus character n-grams (3–5), which cope with agglutination, spelling variants and roman-script typing; vocabulary cap raised about 3× (to ≈ 1,500+ for the complaint corpus; far larger on real data); language field as a feature; per-language evaluation	**Modified**
Retrieval	English thesaurus	Trilingual thesaurus (garbage / kachra / kuppai) for query expansion; a cross-lingual index	**Modified**
Spatial mining	DBSCAN on points, ε = 0.25 km	DBSCAN on cell centres, **ε = 1.01 × cell = 0.253 km** (adjacent cells are neighbours), MinPts = 8; co-location neighbours = same or adjacent cell	**Modified (parameters)**
Reports	Drill-down to street segment	Drill-down to grid cell and ward; language slice; cells with fewer than 5 requests suppressed	**Modified (minor)**
Fact table, Date, Time, Department, Channel, web mining, OLAP	As Task 8	Unchanged	**Retained**
Privacy module	k-anonymity check before release	Mandatory k ≥ 5 gate on every report and extract; l-diversity on complaint type	**Strengthened**

Why modify rather than redesign? The star schema, the technique choices (classification, DBSCAN, co-location, OLAP) and the privacy-in-ETL principle are all still valid; what fails is specific: an English-only vocabulary and exact locations. A full redesign would discard parts that Tasks 1–6 show to work. A language router with three separate models is possible, but Table 17 shows that per-language models reach 1.00 only on their own language (0.25–0.62 elsewhere), so any wrong language detection or code-mixed complaint ("school ke paas pothole") would fail; one multilingual model is simpler to maintain and handles mixing, while the per-language evaluation lets me detect if one language lags and then switch that language to its own model.

D. Evidence-Based Defence

Decision	Evidence from my own tasks	Trade-off accepted
Generalise to a 250 m grid (not exact points, not 500 m or 1 km)	Task 6 data and Table 18: with the 250 m grid DBSCAN keeps 4 clusters, ARI 0.928 against 0.925 for exact points and 97% agreement; at 500 m ARI falls to 0.719 and the strip and the dense blob merge; at 1 km everything is one cluster (ARI 0.0). The k-distance knee at 0.243 km (Figure 4) explains why: hot-spots are about 0.25 km wide, so a cell of that size is the coarsest unit that still resolves them.	**Privacy gain:** a 250 m cell (0.0625 km²) covers many households in a city of 4 million, and the raw GPS is deleted. **Accuracy cost:** a position error of up to 0.177 km (half the cell diagonal), so hot-spots narrower than one cell, or a single junction, can no longer be pinpointed, and noise-labelled points change from 183 to 162. I accept this loss of exactness because the Commissioner needs areas for action, not addresses.
Multilingual model with a larger vocabulary	Task 2 / Table 5: capping the vocabulary at 30 terms cut SVM accuracy from 0.922 to 0.528. Table 17: three languages raise the vocabulary about 2.7× and the capped model drops to 0.672 (cap 30) or 0.891 (cap 60) from 1.00; the English-only model gets 0.458 on Hindi and 0.268 on Tamil.	**Cost:** larger feature matrix, longer training (minutes instead of seconds), and a language-specific resource to maintain (stop-words, stemmers). **Accepted** so that accuracy for Tamil and Hindi citizens is not worse than for English users; this is also a fairness requirement (Task 7, risk 2).
Remove identifiers and mask PII in text	Task 7: the raw extract was 1-anonymous (k = 1); generalising to age bands and PIN prefixes gave k = 3 at the price of ±5 years of age and area-level PIN. Task 8 privacy controls.	**Cost:** a few useful tokens (street names, personal names) are masked, which may lower classifier accuracy slightly (not measured in my experiments; I would measure it on real data before go-live), and small cells (< 5) are hidden in reports. **Accepted:** privacy is a legal requirement, not an option.
Retain the star schema, DBSCAN and the web mining parts	Tasks 3, 5, 6 and 8 do not depend on exact addresses or on English text.	None, which keeps the change small and the risk of new errors low.

Summary of the trade-off. I accept a spatial resolution of 250 m (cost in Task 6 measured as no loss in ARI, 183 → 162 noise points, up to 177 m position error) and a roughly threefold larger text vocabulary, in exchange for a platform that holds no personal identifiers or exact home locations and works for English, Tamil and Hindi complaints. I would **not** go to 500 m, because the measured loss (ARI 0.925 → 0.719) is large, and I would **not** keep an English-only vocabulary, because it

fails for two of the three languages. Limit of this evidence: the multilingual test used a small synthetic roman-script corpus and the clustering used synthetic points, so on real data the figures will differ and the experiments must be repeated before go-live.

APPENDICES – Source Code, Data and Settings

File	Used in	Data, settings and seeds
text_mining.py (Appendix A)	Task 2	Synthetic 600-complaint corpus generated inside the program; random seed 42; stratified 70/30 split; scikit-learn 1.9.1; k-means k = 5, n_init = 10
spatial.py (Appendix B)	Tasks 6 and 9	Synthetic 1,500 GPS points written to incidents.csv; NumPy seed 42; DBSCAN MinPts = 8, ε = 0.25 km; k-means k = 4, n_init = 10
multilingual.py (Appendix C)	Task 9	Synthetic romanised English / Hindi / Tamil complaints, 1,800 documents, seed 42; Linear SVM
incidents.csv	Task 6	1,500 rows: lat, lon, true_group (−1 = noise)

Run with: python3 text_mining.py, python3 spatial.py, python3 multilingual.py (Python 3, scikit-learn, NumPy, SciPy, Matplotlib).

APPENDIX A – text_mining.py (Task 2)

Creates the corpus, the three feature settings, the two classifiers, the confusion matrices and the k-means comparison.

```
"""Task 2 - Text mining on a synthetic Metro City complaint corpus (seed 42)."""
import random, re, json, numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix, precision_recall_fscore_support,
adjusted_rand_score
from sklearn.cluster import KMeans
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
SEED=42; random.seed(SEED); np.random.seed(SEED)
CATS=['Roads','Water','Sanitation','Electricity','Health']
core={
'Roads':['pothole','road damaged','broken footpath','traffic signal not working','speed breaker
missing','road cave in','waterlogging on road','street full of cracks'],
'Water':['water leakage','no water supply','pipeline burst','dirty drinking water','low water
pressure','water tank overflowing','sewage mixing with water','tap water smells bad'],
'Sanitation':['garbage not collected','overflowing dustbin','drain blocked','open dumping','public
toilet dirty','sewage overflow on street','waste burning','stray garbage pile'],
'Electricity':['street light not working','power cut','transformer sparking','loose electric wire','voltage
fluctuation','electric pole leaning','meter box damaged','frequent power failure'],
'Health':['mosquito breeding','dengue cases rising','clinic closed','stagnant water breeding
mosquitoes','no medicine at health centre','fever outbreak in street','food stall unhygienic','dead
animal on road']}
shared=['near school','on main road','in our street','for three days','near bus stand','in ward','near
market','since last week','near temple','near hospital']
filler=['please fix','urgent','kindly act','nobody responded','again and again','residents are
suffering','very bad','complaint registered']
def make(n=120):
    X,y=[],[]
    for c in CATS:
        for _ in range(n):
            parts=[random.choice(core[c])]
            if random.random()<0.25: # ambiguous: borrow from a neighbouring category
                o=random.choice([k for k in CATS if k!=c]); parts.append(random.choice(core[o]))
            parts.append(random.choice(shared))
            if random.random()<0.6: parts.append(random.choice(filler))
            random.shuffle(parts) if random.random()<0.3 else None
            X.append(' '.join(parts)); y.append(c)
    return X,y
X,y=make()
STOP=set('a an the on in at of for near is are not since to and our last please again kindly'.split())  #
small custom list
def stem(w):
    for suf in ('ing','ed','es','s'):
        if w.endswith(suf) and len(w)-len(suf)>=3: return w[:-len(suf)]
    return w
def prep(stop,stm):
    def f(t):
        ws=re.findall(r'[a-z]+',t.lower())
        if stop: ws=[w for w in ws if w not in STOP]
        if stm: ws=[stem(w) for w in ws]
        return ' '.join(ws)
    return f
Xtr,Xte,ytr,yte=train_test_split(X,y,test_size=0.3,random_state=SEED,stratify=y)
settings={
 'S1 binary, no stop-word removal, no stemming, full
vocabulary':dict(kind='bin',stop=False,stem=False,V=5000),
 'S2 TF-IDF, stop-word removal, stemming, full
vocabulary':dict(kind='tfidf',stop=True,stem=True,V=5000),
```

```
 'S3 TF-IDF, stop-word removal, stemming, vocabulary capped at
30':dict(kind='tfidf',stop=True,stem=True,V=30)}
def vec(s):
    pp=prep(s['stop'],s['stem'])
    if s['kind']=='bin': return CountVectorizer(binary=True,preprocessor=pp,max_features=s['V'])
    return TfidfVectorizer(preprocessor=pp,max_features=s['V'])
res={};best=None
for name,s in settings.items():
    v=vec(s); A=v.fit_transform(Xtr); B=v.transform(Xte)
    res[name]={'vocab':len(v.vocabulary_)}
    for cn,clf in (('NaiveBayes',MultinomialNB()),('LinearSVM',LinearSVC(random_state=SEED))):
        clf.fit(A,ytr); p=clf.predict(B); acc=accuracy_score(yte,p)
        res[name][cn]=round(acc,4)
        if best is None or acc>best[0]: best=(acc,name,cn,p,v)
        if name.startswith('S2'):
            cm=confusion_matrix(yte,p,labels=CATS);
pr,rc,f,_=precision_recall_fscore_support(yte,p,labels=CATS,zero_division=0)
            res[name][cn+'_cm']=cm.tolist(); res[name][cn+'_prec']=[round(a,3) for a in pr];
res[name][cn+'_rec']=[round(a,3) for a in rc]
            if cn=='LinearSVM':
                svm_p=p; mis=[(Xte[i],yte[i],p[i]) for i in range(len(yte)) if p[i]!=yte[i]]
                res['misclassified']=mis[:8]; res['n_mis']=len(mis)
            if cn=='NaiveBayes': nb_p=p
# clustering
v=vec(settings['S2 TF-IDF, stop-word removal, stemming, full vocabulary']); M=v.fit_transform(X)
km=KMeans(5,n_init=10,random_state=SEED).fit(M)
ct=np.zeros((5,5),int)
for yy,cc in zip(y,km.labels_): ct[CATS.index(yy),cc]+=1
res['kmeans_table']=ct.tolist(); res['purity']=round(ct.max(0).sum()/len(y),4);
res['ARI']=round(adjusted_rand_score(y,km.labels_),4)
terms=v.get_feature_names_out(); order=km.cluster_centers_.argsort()[:,::-1]
res['cluster_terms']=[[terms[i] for i in order[k,:6]] for k in range(5)]
# a cluster-vs-category majority
res['cluster_major']=[CATS[int(np.argmax(ct[:,k]))] for k in range(5)]

DOM=STOP|set('school road main street bus stand ward market week temple hospital three days
very bad urgent act nobody responded again suffering residents complaint registered fix'.split())
def prep2(t): return ' '.join(stem(w) for w in re.findall(r'[a-z]+',t.lower()) if w not in DOM)
v2=TfidfVectorizer(preprocessor=prep2); M2=v2.fit_transform(X)
km2=KMeans(5,n_init=10,random_state=SEED).fit(M2)
ct2=np.zeros((5,5),int)
for yy,cc in zip(y,km2.labels_): ct2[CATS.index(yy),cc]+=1
res['kmeans2_table']=ct2.tolist(); res['purity2']=round(ct2.max(0).sum()/len(y),4);
res['ARI2']=round(adjusted_rand_score(y,km2.labels_),4)
t2=v2.get_feature_names_out(); o2=km2.cluster_centers_.argsort()[:,::-1]
res['cluster_terms2']=[[t2[i] for i in o2[k,:5]] for k in range(5)]
res['vocab_sizes']={'raw':len(CountVectorizer(preprocessor=prep(False,False)).fit(X).vocabulary_)}
# figure: confusion matrix of SVM
fig,ax=plt.subplots(figsize=(4.8,4))
cm=np.array(res['S2 TF-IDF, stop-word removal, stemming, full vocabulary']['LinearSVM_cm'])
ax.imshow(cm,cmap='Blues'); ax.set_xticks(range(5)); ax.set_yticks(range(5));
ax.set_xticklabels(CATS,rotation=45,ha='right'); ax.set_yticklabels(CATS)
for i in range(5):
    for j in range(5): ax.text(j,i,cm[i,j],ha='center',va='center',color='white' if cm[i,j]>cm.max()/2 else
'black')
ax.set_xlabel('Predicted'); ax.set_ylabel('Actual'); ax.set_title('Linear SVM confusion matrix (S2)');
plt.tight_layout(); plt.savefig('fig_cm.png',dpi=160)
# vocab-size effect incl. multilingual simulation (for Task 9): add Tamil/Hindi transliterated copies
json.dump(res,open('text_results.json','w'),indent=1)
print(json.dumps(res,indent=1)[:6000])
```

APPENDIX B – spatial.py (Tasks 6 and 9)

Generates the incident data, runs k-means and DBSCAN, the k-distance graph, the effect of ε and the grid-generalisation test.

```
"""Task 6 - Spatial clustering of synthetic incident GPS data (seed 42) + Task 9 grid-generalisation test."""
import numpy as np, json
from sklearn.cluster import KMeans, DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import adjusted_rand_score
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
rng=np.random.default_rng(42)
# km coordinates in a 10 x 10 km window; origin (12.95N, 80.15E)
A=rng.normal([2.5,7.0],0.45,(400,2))
t=rng.uniform(0,1,350); B=np.c_[1.0+7.0*t, 2.0+1.2*t]+rng.normal(0,0.10,(350,2))      # elongated road strip
th=rng.uniform(np.pi*0.1,np.pi*1.1,350);
C=np.c_[7.0+1.5*np.cos(th),6.5+1.5*np.sin(th)]+rng.normal(0,0.09,(350,2))  # crescent
D=rng.normal([8.3,2.0],0.22,(200,2))
N=rng.uniform(0,10,(200,2))
P=np.vstack([A,B,C,D,N]);
truth=np.r_[np.zeros(400),np.ones(350),2*np.ones(350),3*np.ones(200),-1*np.ones(200)].astype(int)
lat=12.95+P[:,1]/111.0; lon=80.15+P[:,0]/(111.0*np.cos(np.radians(12.95)))
np.savetxt('incidents.csv',np.c_[lat,lon,truth],delimiter=',',header='lat,lon,true_group',comments='',fmt='%.6f,%.6f,%d')
out={'n':len(P)}
cols=['tab:red','tab:blue','tab:green','tab:orange','tab:purple','tab:brown','tab:pink']
def plot(ax,X,lab,title):
    for l in sorted(set(lab)):
        m=lab==l
        ax.scatter(X[m,0],X[m,1],s=4,c='lightgrey' if l==-1 else cols[l%7],label='noise' if l==-1 else f'C{l}')
    ax.set_title(title,fontsize=9); ax.set_aspect('equal'); ax.set_xlabel('x (km)'); ax.set_ylabel('y (km)')
# k-means
km=KMeans(4,n_init=10,random_state=42).fit(P);
out['kmeans_ARI']=round(adjusted_rand_score(truth,km.labels_),3)
# ARI on non-noise truth only
m=truth!=-1; out['kmeans_ARI_core']=round(adjusted_rand_score(truth[m],km.labels_[m]),3)
# k-distance
MINPTS=8
nn=NearestNeighbors(n_neighbors=MINPTS).fit(P); d,_=nn.kneighbors(P); kd=np.sort(d[:,-1])
# knee: max distance from chord
x=np.arange(len(kd)); xn=x/x[-1]; yn=(kd-kd[0])/(kd[-1]-kd[0]); knee=np.argmax(xn-yn);
eps0=float(kd[knee]); out['knee_eps']=round(eps0,3)
fig,ax=plt.subplots(figsize=(5,3.4)); ax.plot(kd); ax.axhline(0.25,ls='--',c='r'); ax.set_xlabel('points sorted by distance'); ax.set_ylabel(f'{MINPTS}-th NN distance (km)'); ax.set_title('k-distance graph (MinPts = 8)'); plt.tight_layout(); plt.savefig('fig_kdist.png',dpi=160); plt.close()
out['kdist_percentiles']={p:round(float(np.percentile(kd,p)),3) for p in (50,75,80,85,90,95)}
res={}
for eps in (0.10,0.25,0.50,1.00):
    db=DBSCAN(eps=eps,min_samples=MINPTS).fit(P); lab=db.labels_
    res[eps]={'clusters':int(len(set(lab))-(1 in [])- (1 if -1 in lab else 0)),'noise':int((lab==-1).sum()),'ARI':round(adjusted_rand_score(truth,lab),3)}
    # noise recovered: fraction of true-noise points labelled -1
    res[eps]['true_noise_found']=round(float(((lab==-1)&(truth==-1)).sum()/(truth==-1).sum()),3)
out['dbscan_eps']={str(k):v for k,v in res.items()}
EPS=0.25; db=DBSCAN(eps=EPS,min_samples=MINPTS).fit(P); lab=db.labels_
out['dbscan_chosen']={'eps':EPS,'minpts':MINPTS,'clusters':int(len(set(lab))-(1 if -1 in lab else 0)),'noise':int((lab==-1).sum()),'ARI':round(adjusted_rand_score(truth,lab),3),'sizes':{int(l):int((lab==l).sum()) for l in sorted(set(lab))}}
fig,axs=plt.subplots(1,2,figsize=(9,4.2)); plot(axs[0],P,km.labels_,'k-means (k = 4)');
plot(axs[1],P,lab,f'DBSCAN (eps = {EPS} km, MinPts = {MINPTS})');
```

```
axs[1].legend(markerscale=3,fontsize=7); plt.tight_layout();
plt.savefig('fig_kmeans_dbscan.png',dpi=160); plt.close()
fig,axs=plt.subplots(1,4,figsize=(13,3.4))
for ax,eps in zip(axs,(0.10,0.25,0.50,1.00)):
    l=DBSCAN(eps=eps,min_samples=MINPTS).fit(P).labels_; plot(ax,P,l,f'eps = {eps} km: {len(set(l))-(1 if -1 in l else 0)} clusters, {int((l==-1).sum())} noise')
plt.tight_layout(); plt.savefig('fig_eps_effect.png',dpi=160); plt.close()
# Task 9: grid generalisation
g={}
fig,axs=plt.subplots(1,4,figsize=(13,3.4))
plot(axs[0],P,lab,'Exact GPS (eps 0.25)')
for ax,cell in zip(axs[1:],(0.25,0.5,1.0)):
    Q=(np.floor(P/cell)+0.5)*cell
    best=None
    for epsg in (cell*1.01,cell*1.5):
        l=DBSCAN(eps=epsg,min_samples=MINPTS).fit(Q).labels_
        r={'cell_km':cell,'eps':round(epsg,3),'clusters':int(len(set(l))-(1 if -1 in l else 0)),'noise':int((l==-1).sum()),'ARI_vs_truth':round(adjusted_rand_score(truth,l),3),'agree_with_exact':round(adjusted_rand_score(lab,l),3),'unique_cells':int(len({tuple(q) for q in Q}))}
        if best is None or r['ARI_vs_truth']>best[0]['ARI_vs_truth']: best=(r,l)
        g.setdefault(str(cell),[]).append(r)
    plot(ax,Q,best[1],f'{int(cell*1000)} m grid, eps {best[0]["eps"]}')
plt.tight_layout(); plt.savefig('fig_grid_effect.png',dpi=160); plt.close()
out['grid']=g
json.dump(out,open('spatial_results.json','w'),indent=1); print(json.dumps(out,indent=1))
```

APPENDIX C – multilingual.py (Task 9)

Simulates English, Hindi and Tamil (romanised) complaints and tests the English-only and multilingual classifiers.

```
"""Task 9 - effect of three languages on vocabulary and classifier (romanised, synthetic, seed 42)."""
import random, re, json, numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
random.seed(42)
CATS=['Roads','Water','Sanitation','Electricity','Health']
P={ # (English, Hindi, Tamil) - romanised, illustrative
'Roads':[('pothole','sadak par gaddha','saalaiyil pallam'),('broken footpath','toota footpath','udaindha nadaipaadhai'),('traffic signal not working','signal kharab hai','signal velai seyyavillai')],
'Water':[('water leakage','paani ka risaav','thanneer geeral'),('no water supply','paani ki supply nahi','thanneer viniyogam illai'),('dirty drinking water','ganda peene ka paani','asuthamaana kudineer')],
'Sanitation':[('garbage not collected','kachra nahi uthaya','kuppai edukkavillai'),('drain blocked','naali band hai','kaalvai adaippu'),('overflowing dustbin','kachredaan bhar gaya','kuppai thotti nirambi')],
'Electricity':[('street light not working','street light kharab','theru vilakku eriyavillai'),('power cut','bijli ki katauti','minsaaram thadai'),('loose electric wire','dheela bijli taar','thalarntha minkambi')],
'Health':[('mosquito breeding','machhar panapna','kosu uruvaakkam'),('dengue cases rising','dengue ke mamle badh rahe','dengu paravugiradhu'),('clinic closed','clinic band hai','maruthuvamanai moodapattullathu')]}
LOC=[('near school','school ke paas','palli arugil'),('on main road','mukhya sadak par','mukkiya saalaiyil'),('near market','bazaar ke paas','sandhai arugil')]
rows=[]
for c in CATS:
    for _ in range(120):
        for li,lang in enumerate(('en','hi','ta')):
            k=random.choice(P[c]); l=random.choice(LOC); rows.append((k[li]+' '+l[li],c,lang))
random.shuffle(rows)
X=[r[0] for r in rows]; y=[r[1] for r in rows]; L=[r[2] for r in rows]
idx=np.arange(len(X)); tr,te=train_test_split(idx,test_size=0.3,random_state=42,stratify=y)
tok=lambda t:re.findall(r'\w+',t.lower())
def run(trmask,V=None):
    v=TfidfVectorizer(tokenizer=tok,token_pattern=None,max_features=V); A=v.fit_transform([X[i] for i in trmask]); clf=LinearSVC(random_state=42).fit(A,[y[i] for i in trmask])
    out={}
    for lg in ('en','hi','ta'):
        te_l=[i for i in te if L[i]==lg]; out[lg]=round(accuracy_score([y[i] for i in te_l],clf.predict(v.transform([X[i] for i in te_l]))),3)
    out['all']=round(accuracy_score([y[i] for i in te],clf.predict(v.transform([X[i] for i in te]))),3); out['vocab']=len(v.vocabulary_); return out
R={}
R['english_only_train']=run([i for i in tr if L[i]=='en'])
R['multilingual_train_full_vocab']=run(list(tr))
R['multilingual_train_vocab_30']=run(list(tr),30)
R['multilingual_train_vocab_60']=run(list(tr),60)
for lg in ('en','hi','ta'): R['per_language_model_'+lg]=run([i for i in tr if L[i]==lg])
R['vocab_by_language']={lg:len({w for i,t in enumerate(X) if L[i]==lg for w in tok(t)}) for lg in ('en','hi','ta')}
json.dump(R,open('multilingual_results.json','w'),indent=1); print(json.dumps(R,indent=1))
```